

Contents

1	Introduction	4
1.1	Problem Description	4
1.2	Mathematical Formulation	6
1.2.1	Modelling Framework	6
1.2.2	Problem Definition	6
1.2.3	Model Definition	6
2	Instance Generation	9
2.1	Creating Instances	9
2.1.1	Configuration-Driven Design	9
2.1.2	Component-Based Construction	9
2.1.3	Board Constraints	10
2.1.4	Stochastic Placement and Feasibility Checks	10
2.2	Storing Instances	10
2.2.1	Board Representation	11
2.2.2	Cost Matrix Representation	11
2.3	Loading Instances	11
2.4	Examples of Generated Instances	12
2.4.1	Random Layouts	12
2.4.2	Structured Component Layouts	12
2.5	Instance Configuration Parameters	13
2.5.1	Board	13
2.5.2	Components	14
2.5.3	Reproducibility and Custom Instances	14
3	Model Implementation	15
3.1	Input Interface and Data Preparation	15
3.2	SCF Solver Implementation	16
3.2.1	Decision Variables and Objective Function	16
3.2.2	Constraints	18
3.3	Solution Extraction and Interpretation	25
3.4	Running the SCF Solver	26
3.4.1	Compilation	26
3.4.2	Solving an Instance	27
3.4.3	Batch Solver	28
3.5	Sanity Check for Model Correctness	28
4	Heuristic Algorithm	30
4.1	Local Search Algorithm Design	30
4.1.1	Solution Representation	30

4.1.2	Finding Initial Solutions	31
4.1.3	Defining the Neighbourhood of a Solution	32
4.1.4	Evaluating the Quality of a Solution	32
4.1.5	Exploration Strategies: Selecting the Next Solution	33
4.2	A First Attempt to Metaheuristics: Tabu Search	34
4.2.1	Setting Up the Tabu List	34
4.2.2	Defining the Aspiration Criteria	35
4.2.3	Dynamic Tabu Search Behavior	35
4.2.4	Improving the Performance: Multi Start & Parallelization	36
4.3	Exploiting the Problem Structure	37
4.3.1	Clustering Algorithm Selection	37
4.4	A Second Attempt: Hierarchical Tabu Search	41
4.4.1	Inter-Cluster Optimization	41
4.4.2	Intra-Cluster Optimization	42
4.4.3	Stitching the Cluster Tours to Obtain a Global Tour	42
4.4.4	Polishing the Stitched Global Tour	44
4.4.5	Parameter Configuration & Tuning	45
4.5	Running the Heuristic Solver	46
4.5.1	Compilation	46
4.5.2	Solving an Instance	46
4.5.3	Batch Solver	47
5	Results	48
5.1	Benchmark Instances and Experiments	48
5.2	CPLEX SCF Solver Performance	49
5.2.1	Execution Times	49
5.2.2	Comparing Metrics	52
5.3	Heuristic Algorithms Performance	53
5.3.1	Speedup & (Pessimistic) Optimality Gaps	54
5.3.2	The Impact of Parallelization and Structural Decomposition	56
5.4	Comparing SCF Solver and Our Metaheuristics	59
5.4.1	Qualitative Comparison of Running Times & Optimality Gaps	59
5.4.2	Using Exact Solvers as the Comparison Baseline	59
5.5	Beyond Numbers: Tour Structure Comparison	61
5.5.1	Qualitative Analysis of Tours	61
5.6	Possible Improvements	62
5.6.1	Enhanced Local Search Operators	62
5.6.2	Reactive and Adaptive Parameter Tuning	62
5.6.3	Improving Clustering to Mitigate the Price of Hierarchy	63
5.6.4	Integrating Exact Methods	63
5.6.5	Implementation-Level Improvements	63
5.7	Conclusions	65

This chapter introduces the problem, beginning with a real-world scenario. We then present a graph-based framework for its formal definition and conclude by developing a mathematical model that specifies the objective and the constraints.

1.1 Problem Description

A company manufactures perforated boards used in the assembly of electrical panels. During production, each board is mounted on a drilling machine where a drill head moves across the surface, stops at predefined positions, and drills the required holes. Once a board is completed, it is replaced by the next one, and the process repeats continuously.

Given a set of hole positions on a board, the objective is to determine an ordering of drilling operations that minimizes the total processing time. Since the drilling time at each hole is assumed to be constant, minimizing the overall processing time reduces to minimizing the travel time of the drill head between successive hole locations.

The travel time between two holes depends on the motion characteristics of the drilling head. In particular, the machine may operate at different maximum speeds along each axis, and the way movements along the axes are coordinated directly affects how long it takes to move from one position to another. As a result, travel times cannot be inferred solely from geometric distances, but must instead be computed explicitly for each ordered pair of holes.

The kinematic configuration of the drilling machine therefore induces a specific distance metric used to evaluate travel times. In this work, we consider three representative movement modes:

1. Sequential axis movement (L_1 norm): the drill head moves along one axis at a time. In this case, the travel time is proportional to the Manhattan distance between two points. This mode reflects a system in which motion along one axis precludes motion along the other.
2. Coordinated straight-line movement (L_2 norm): the drill head follows a straight-line path between source and destination. This corresponds to the Euclidean distance, under the assumption that maintaining a fixed movement direction requires sharing power between axes, thereby limiting achievable speed.
3. Simultaneous independent axis movement (L_∞ norm): the machine is equipped with independent motors that can operate simultaneously at their maximum speeds along both axes. In this case, the travel time is determined by the axis requiring the longest time to complete its motion, corresponding to the Chebyshev distance.

To clarify the practical differences between these metrics, consider the plots in Figure 1, which visually demonstrate how the drill head navigates space under each kinematic constraint.

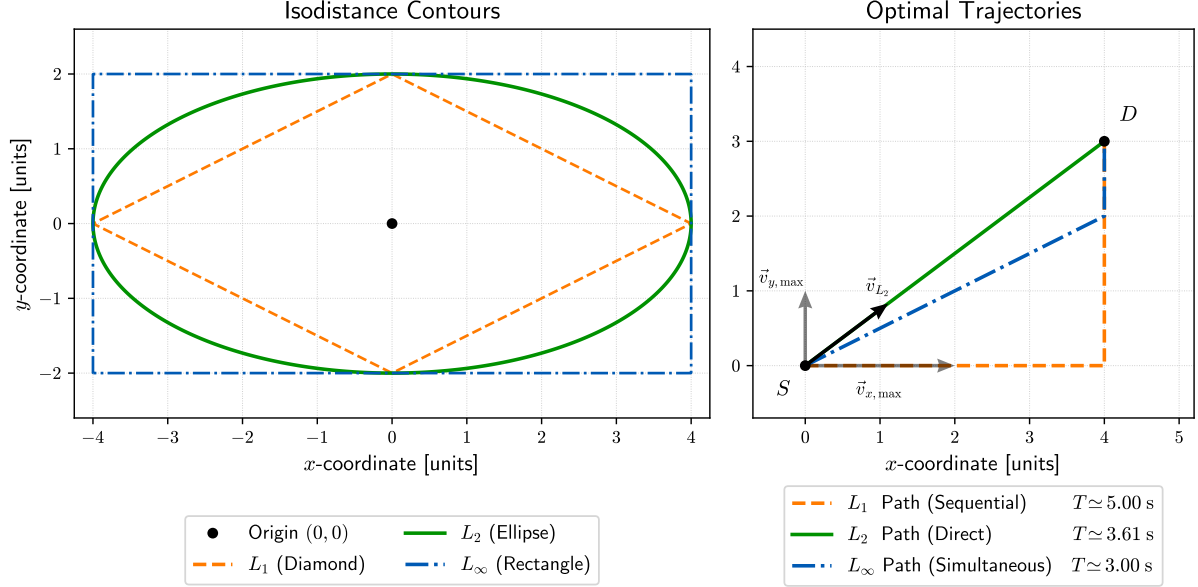


Figure 1: Comparison of different distance metrics, in terms of trajectories and reachable distances.

The left plot (Isodistance Contours) illustrates the region reachable from a fixed origin within a unit time under each metric. The L_1 norm yields the smallest reachable area, reflecting the inefficiency of sequential axis movement, where one axis remains idle while the other moves. The L_∞ norm produces the largest reachable area, as both axes operate independently at full speed. The L_2 norm lies between these extremes, since the straight-line constraint imposes a shared power limitation.

The right plot (Optimal Trajectories) shows example paths from a source S to a destination D . Under the L_1 distance metric, movement occurs in two distinct phases (one axis at a time), resulting in the longest travel time. The L_2 distance metric produces a straight-line trajectory with intermediate arrival time due to speed constraints. Under the L_∞ distance metric both axes move simultaneously at maximum speed; although the geometric path may differ from the Euclidean straight line, the arrival time is minimized and depends only on the slower axis.

It is important to note that these kinematic considerations affect only the preprocessing stage of the problem, specifically the computation of the travel-time cost matrix. Once the pairwise travel times have been determined, the optimization model itself is agnostic to the underlying distance metric. Consequently, the choice of distance metric serves to accurately represent the physical behavior of the drilling machine, rather than to alter the structure of the optimization model.

Finally, without loss of generality (and for simplicity), we assume all instances to be symmetric. That is, the travel time from hole i to hole j is equal to the travel time from j to i . This assumption not only reflects realistic drilling scenarios under the kinematic

configurations described above, but also allows the heuristic algorithms to explicitly exploit symmetry in both their design and implementation, resulting in reduced memory usage and improved computational efficiency.

1.2 Mathematical Formulation

1.2.1 Modelling Framework

The generic instance of the problem can be modeled as a weighted complete graph, denoted as $G = (\mathcal{N}, \mathcal{A})$, where $\mathcal{N}$ is the set of nodes representing the holes that the drilling machine must process, and $\mathcal{A}$ is the set of arcs connecting each pair of nodes.

Formally, we can define $\mathcal{N} = \{0, 1, \dots, n-1\}$ and $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N} \wedge i \neq j\}$, with a function $c : \mathcal{A} \rightarrow \mathbb{R}$ that maps $(i, j) \in \mathcal{A} \mapsto c(i, j)$, representing the cost of the trajectory between nodes i and j . This cost can be interpreted as the time required for the drill head to move from i to j , i.e. $c(i, j) = \text{time}(i \rightarrow j)$. The condition $i \neq j$ for the set $\mathcal{A}$ of arcs naturally arises in the context of our problem, since we can exclude loops, i.e. arcs that go from one node to itself. This is reasonable because the drill head moves between nodes, with no benefit gained from remaining at the same position.

1.2.2 Problem Definition

With the framework established, we can now proceed to formally define the problem.

The Travelling Drillman's Holey Grail

Let $G = (\mathcal{N}, \mathcal{A})$ denote the graph representing an instance of our problem, in the form provided by the framework described in 1.2.1. Thus, solving the problem reduces to finding a minimum-cost path that starts at a node, visits all other nodes exactly once, and returns to the starting node. In other words, we seek a Hamiltonian cycle in G , which is equivalent to solving the Travelling Salesman Problem (TSP) on G .

In line with the model we have defined, we assume that the drill head returns to the initial hole after completing all others. This assumption is reasonable, as it ensures that the drill head is ready to start drilling the next board.

1.2.3 Model Definition

Several formulations for the TSP have been formulated over time using Mixed Integer Linear Programming (MILP). Some of them include an exponential number of constraints and in practice they cannot be directly implemented using basic CPLEX APIs. For this reason, we consider a more compact formulation that requires a polynomial (possibly with small degree) number of variables and constraints. More specifically, we consider a compact formulation based on network flows (Single-Commodity Flow formulation).

Given the graph $G = (\mathcal{N}, \mathcal{A})$ for an instance of our problem, we arbitrarily choose a node in $\mathcal{N}$ as the starting node (denoted as node 0 for simplicity). Now, let $n = |\mathcal{N}|$, and

let the starting node send out a total flow of $n - 1$. The idea is then to route this flow to the remaining nodes in a way such that (i) each node other than the starting node consumes exactly one unit of flow, (ii) each node is visited exactly once and (iii) the total cost of all arcs carrying flow is minimized. This high level definition of the model can be formalized as follows, by defining

Sets:

- $\mathcal{N} = \{0, 1, \dots, n - 1\}$: graph nodes.
- $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N} \wedge i \neq j\}$: graph arcs.

Parameters:

- $c_{ij} = c(i, j) = \text{time}(i \rightarrow j)$ for all $(i, j) \in \mathcal{A}$.
- $0 \in \mathcal{N}$: the starting node (arbitrarily selected).

Decision Variables:

- $x_{ij} = \text{flow}(i \rightarrow j)$: amount of flow shipped from i to j , for all $(i, j) \in \mathcal{A}$.
- $y_{ij} : 1$ if $(i, j) \in \mathcal{A}$ carries some flow, 0 otherwise, for all $(i, j) \in \mathcal{A}$.

Here, $c_{ij}, x_{ij} \in \mathbb{R}_{\geq 0}$ and $y_{ij} \in \{0, 1\}$. Note that the two sets of variables are linked: whenever $y_{ij} = 0$, it must be the case that $x_{ij} = 0$. This dependency will be enforced through a specific constraint in the final model.

We can now formally define the objective function z to be minimized, according to (iii):

$$z = \sum_{\substack{i,j \\ (i,j) \in \mathcal{A}}} c_{ij} y_{ij} . \quad (1.1)$$

At this point, we only need to introduce the constraints to complete the model. To begin with, we need to enforce the condition that the starting node (source) ships exactly $n - 1$ units of flow:

$$\sum_{j: (0,j) \in \mathcal{A}} x_{0j} = n - 1. \quad (1.2)$$

Now we enforce (i) for all other nodes (sinks), which requires

$$\sum_{i: (i,k) \in \mathcal{A}} x_{ik} - \sum_{j: (k,j) \in \mathcal{A}} x_{kj} = 1 \quad \forall k \in \mathcal{N} \setminus \{0\}. \quad (1.3)$$

Next, we impose the degree constraints that encode condition (ii):

$$\begin{aligned} \sum_{i: (i,j) \in \mathcal{A}} y_{ij} &= 1 \quad \forall j \in \mathcal{N}, \\ \sum_{j: (i,j) \in \mathcal{A}} y_{ij} &= 1 \quad \forall i \in \mathcal{N}, \end{aligned} \quad (1.4)$$

which ensure each node is entered exactly once and left exactly once (so each node receives its flow from a single incoming arc and sends its flow along a single outgoing arc).

Finally, we need to link the variables and state their domains. We have

$$x_{ij} \leq y_{ij}(n-1) \quad \forall (i, j) \in \mathcal{A}, \quad (1.5)$$

which forces $x_{ij} = 0$ whenever $y_{ij} = 0$. The variable domains are $y_{ij} \in \{0, 1\} \quad \forall (i, j) \in \mathcal{A}$ and $x_{ij} \in \mathbb{R}_{\geq 0} \quad \forall (i, j) \in \mathcal{A}$. Before showing the full model, we notice that the constraint (1.2) is redundant. Indeed, constraint (1.3) implicitly enforces the lower bound, as at least $n-1$ unit of flows must be available. In addition to this, no more than $n-1$ units of flow will be shipped by the starting node (source), due to the constraints in (1.4) (only one outgoing arc can be selected for each node, including the source) and in (1.5) (the maximum flow going through the selected outgoing arc is at most $n-1$). Finally, note that there is no need of shipping flow towards the source node, which means we can fix $x_{i0} = 0$ for all $i \in \mathcal{N}$ (i.e. we can eliminate those variables).

The full model reads as follows:

$$\begin{aligned} \min \quad & \sum_{\substack{i,j \\ (i,j) \in \mathcal{A}}} c_{ij} y_{ij} \\ \text{s.t.} \quad & \sum_{i:(i,k) \in \mathcal{A}} x_{ik} - \sum_{\substack{j:(k,j) \in \mathcal{A} \\ j \neq 0}} x_{kj} = 1 \quad \forall k \in \mathcal{N} \setminus \{0\} \\ & \sum_{i:(i,j) \in \mathcal{A}} y_{ij} = 1 \quad \forall j \in \mathcal{N} \\ & \sum_{j:(i,j) \in \mathcal{A}} y_{ij} = 1 \quad \forall i \in \mathcal{N} \\ & x_{ij} + (1-n)y_{ij} \leq 0 \quad \forall (i, j) \in \mathcal{A}, \quad j \neq 0 \\ & x_{ij} \in \mathbb{R}_{\geq 0} \quad \forall (i, j) \in \mathcal{A}, \quad j \neq 0 \\ & y_{ij} \in \{0, 1\} \quad \forall (i, j) \in \mathcal{A} \end{aligned}$$

Note that we arranged the terms so that all variables appear on the LHS and the values on the RHS. This will be especially useful when translating the model into a proper implementation using the CPLEX APIs.

To interpret the resulting solution, it is important to distinguish between the roles of the two variable sets. The binary variables y_{ij} for $(i, j) \in \mathcal{A}$ define the physical topology of the tour, indicating whether the drill head travels directly between two specific nodes (holes). Conversely, the continuous flow variables x_{ij} for $(i, j) \in \mathcal{A}$ represent a fictitious commodity used solely to enforce connectivity and eliminate subtours. While they are mathematically essential for the validity of the single-commodity flow formulation, they act merely as support variables in the final output. For this reason, the operational drilling sequence is fully determined by the active y_{ij} arcs.

With the theoretical framework established, we can now move to the implementation.

This chapter presents the instance generation process, starting from the real-world problem and progressively translating it into a formal representation. We first provide a high-level description of the structure and parameters defining an instance, then detail how instances are generated based on these parameters. Finally, we introduce the file format used to store instances, ensuring consistency and ease of integration with the solver.

2.1 Creating Instances

To rigorously evaluate the proposed optimization model, we developed a custom instance generator designed to reflect the structural characteristics of real industrial circuit boards. Since there are infinitely many ways to generate drilling layouts, the generator enforces a set of controlled constraints that balance realism, diversity, and reproducibility.

Rather than modeling boards as unstructured collections of random points, instances are constructed to mimic the spatial organization induced by electronic components. In real boards, holes are rarely placed arbitrarily; instead, they appear in structured patterns dictated by component footprints, mechanical constraints, and assembly requirements. The generator explicitly incorporates this structure.

2.1.1 Configuration-Driven Design

The generation process is fully configuration-driven. Each dataset is specified through a declarative configuration file that defines:

- the physical dimensions of the board,
- the safety constraints such as edge margins and minimum hole spacing,
- the presence of fixed mechanical features (e.g. mounting holes),
- and the types and quantities of components to be placed.

This approach enables reproducibility while allowing the systematic exploration of diverse scenarios, ranging from sparse and loosely structured layouts to dense and highly organized boards. As a result, the optimization algorithms can be tested across a wide spectrum of instance characteristics.

2.1.2 Component-Based Construction

At the core of the generator lies a component-based constructive approach. Instead of treating each hole as an independent entity, holes are generated in structured groups corresponding to common component footprints. Each component defines a local geometric pattern, which is then placed onto the board as a single unit. More specifically, the following component primitives are supported:

- Grids: representing regular pin layouts such as those found in integrated circuits or sockets.
- Linear arrays (straight lines): modeling connector headers or strip terminals.
- Random holes: used to introduce controlled irregularity and represent discrete components such as resistors or capacitors.
- and the types and quantities of components to be placed.

By assembling instances from these primitives, the resulting layouts preserve local spatial correlations commonly observed in real production boards. This provides a significantly more realistic challenge for drill-path optimization than purely random point sets.

2.1.3 Board Constraints

Before placing variable components, the generator defines the static constraints of the physical board. A rectangular canvas with fixed width and height is established, together with a safety margin that prevents drilling too close to the board edges, where mechanical clamping or material instability may occur.

Optionally, fixed mounting holes can be generated at predetermined locations such as corners, edges, or the board center. These holes represent mechanical fixation requirements and are treated as deterministic features, independent of the component placement logic.

2.1.4 Stochastic Placement and Feasibility Checks

Components are placed using a stochastic try-and-check procedure. For each component instance, the generator:

1. Constructs the component's local hole pattern.
2. Applies a rotation, either fixed or randomly sampled.
3. Samples a translation within the valid board area.
4. Verifies physical feasibility before final placement.

Feasibility is enforced through collision detection: the Euclidean distance between every candidate hole and all previously placed holes must exceed a configurable minimum spacing threshold. This threshold reflects the drill bit diameter plus a safety tolerance. If a placement violates this constraint, it is rejected and resampled, up to as many times as specified by a proper parameter, as to avoid infinite looping.

This process ensures that all generated instances are not only mathematically valid, but also physically manufacturable, avoiding unrealistically dense or overlapping layouts.

2.2 Storing Instances

After generating the board layouts, instances must be stored in a consistent and structured format to allow reliable reuse. This is essential both for flexibility of the optimization framework and for reproducibility, as the same instance can be processed multiple times to

compute averages and obtain statistically robust results. The storage pipeline is organized into two distinct stages.

2.2.1 Board Representation

In the first stage, each generated board is stored as a standalone **JSON** file. This file encodes the geometric and structural description of the instance and includes:

- the board dimensions and safety constraints,
- the complete list of hole coordinates (x, y) for the components that identify all holes on the board, and
- metadata such as measurement units and unique instance identifiers.

At this stage, instances are purely geometric. The placement of holes is represented only by their coordinates within the board, without any assumptions about the drill head’s motion or travel-time model.

2.2.2 Cost Matrix Representation

In the second stage, a dedicated preprocessing step converts the geometric instance into a weighted routing problem. Given a selected distance metric and the motion parameters of the drill head, the processor computes the full travel-time cost matrix between all ordered pairs of holes.

The conversion from geometry to routing costs is performed with respect to the node identifiers defined in the geometric instance. In the **JSON** representation, each hole is associated with a unique integer **id**, which establishes a fixed ordering of nodes. This ordering is preserved during preprocessing and used consistently to index the cost matrix: the entry in row i and column j corresponds to the computed travel time from the hole with **ID** i to the hole with **ID** j . In particular, node **IDs** are assumed to range from 0 to $n - 1$, where n is the number of holes, ensuring a one-to-one correspondence between geometric nodes and matrix indices.

This cost matrix captures all kinematic effects, including axis-dependent speeds, asymmetries, and the chosen movement model. The resulting matrix is stored in a separate **.dat** file.

2.3 Loading Instances

The optimization algorithm operates exclusively on the precomputed cost matrix and has no access to geometric coordinates, board dimensions, or kinematic parameters. From the solver’s perspective, the problem is therefore fully specified by the matrix alone, which cleanly decouples instance generation, physical and kinematic modeling, and optimization. This design allows different motion models or distance metrics to be evaluated on the same geometric instance without modifying the optimization code, while also enabling the same cost matrix to be reused across multiple experimental runs.

2.4 Examples of Generated Instances

To conclude the chapter and to illustrate the flexibility of the proposed instance generator, we present a selection of representative board layouts with increasing structural complexity.

2.4.1 Random Layouts

The simplest instances consist solely of randomly placed holes, subject only to edge margins and minimum spacing constraints (and possibly mounting holes). These layouts exhibit little to no spatial structure and serve as baseline cases for evaluating algorithmic performance under near-uniform randomness. Figure 2 shows some examples of such layouts.

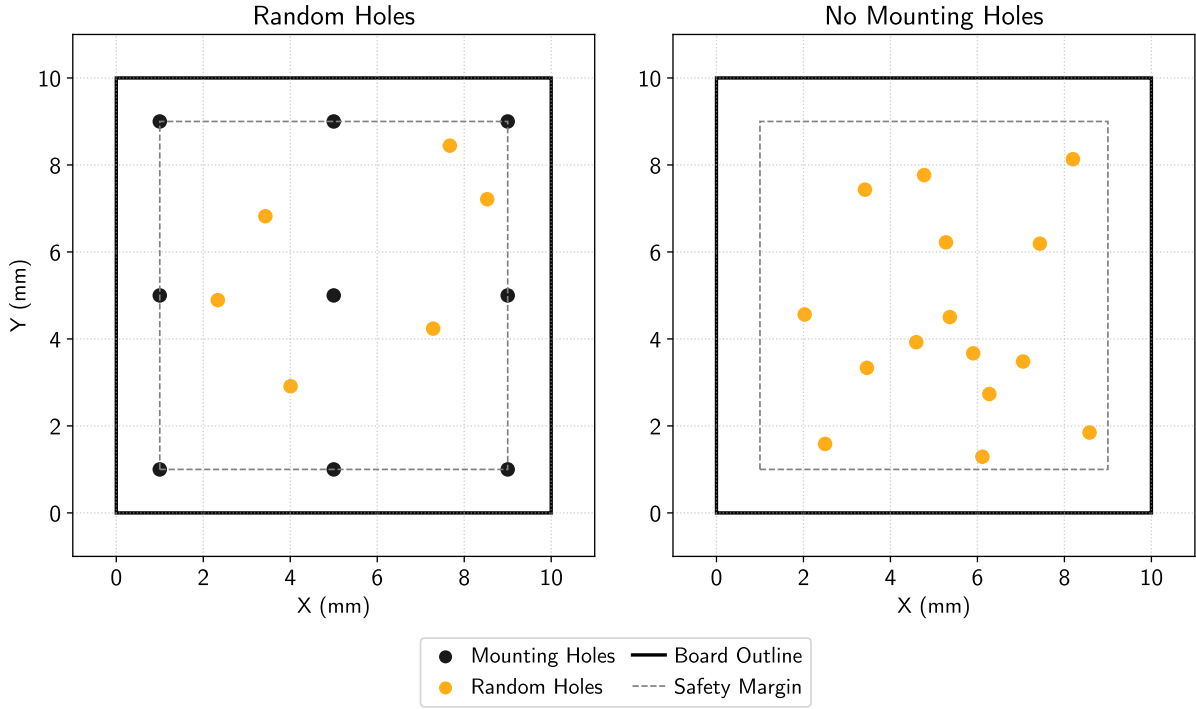


Figure 2: Board Instances with random unstructured hole placement.

2.4.2 Structured Component Layouts

More realistic instances are obtained by composing structured components such as grids and linear arrays, which reflect common placement patterns in industrial boards. Components may vary in size, orientation, spacing, and placement, producing dense and complex layouts.

The resulting hole distributions exhibit strong local clustering and regular spacing, introducing geometric correlations that shape high-quality solutions. Compared to fully random instances, these layouts are more combinatorially challenging but also more amenable to exploiting regularity: structure-aware heuristics can construct near-optimal traversal strategies in reasonable time, even for large instances where exact approaches based on the SCF formulation in CPLEX become computationally demanding. Since the solver operates solely on the numerical cost matrix and does not exploit geometric patterns explicitly, heuristics can capitalize on these correlations to achieve strong performance as instance size grows. Some examples of structured instances are shown in Figure 3.

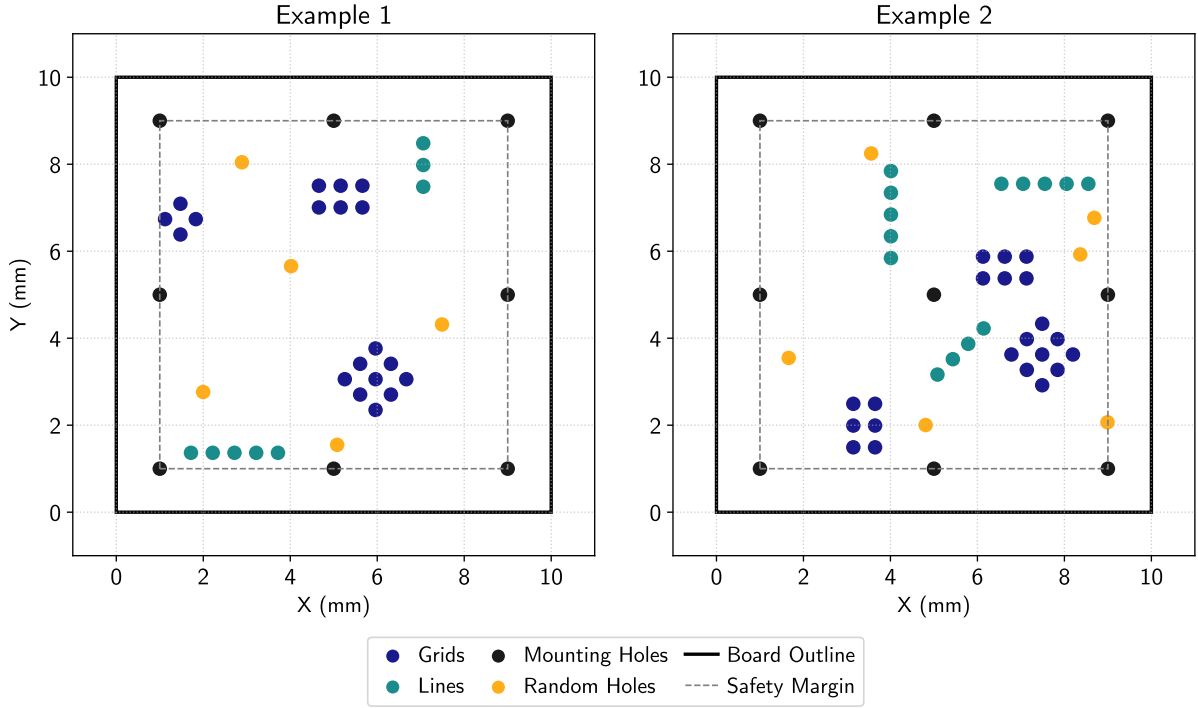


Figure 3: Board Instances with structured hole placement using components.

2.5 Instance Configuration Parameters

To facilitate the creation of custom instances, this section summarizes the configuration parameters associated with the board and with each component type supported by the instance generator. By modifying these parameters, you can easily construct board layouts with different structural characteristics without changing the underlying generation logic. All components are specified declaratively in the `board_layouts.yaml` configuration file.

2.5.1 Board

The following table summarizes the global board-level parameters defined in the configuration file, which establish the physical dimensions, safety constraints, and fixed structural features shared by all generated instances.

Parameter Name	Parameter Description
<code>width, height</code>	Board dimensions
<code>margin</code>	Safety margin from the board edges
<code>min_spacing</code>	Minimum distance between any two holes
<code>mounting_holes</code>	Composite flag for controlling fixed mounting holes

Table 1: YAML configuration parameter keys for the board.

The `mounting_holes` entry controls the optional generation of fixed mechanical holes used for board fixation. When enabled, mounting holes may be placed at predefined locations, including the board corners, the midpoint of each edge, and the board center. Each of these locations can be independently activated or deactivated through appropriate subkeys.

2.5.2 Components

Once the global board constraints are defined, the configuration specifies the individual components to be placed on the board, each characterized by its own geometric parameters and placement rules. These rules are summarized in the table that follows. Additionally, all component types share the parameter **repeat** that controls how many times the corresponding component time is instantiated (and placed somewhere in the board, if possible). This allows for generating multiple components of the same type without repeating the configuration each time.

Component Type	Parameters
Grid	<code>rows</code> , <code>cols</code> , <code>row_spacing</code> , <code>col_spacing</code> , <code>angle_deg</code>
Line	<code>n_holes</code> , <code>spacing</code> , <code>angle_deg</code>
Random	<code>n_holes</code>

Table 2: **YAML** configuration parameter keys for the geometric components.

It is important to note that even if a line component could be represented as a grid with a single row and an appropriate number of columns, we chose to model lines as a distinct component type. This separation improves clarity at the configuration level and simplifies downstream processing. In particular, it facilitates visualization, where grids and lines can be rendered using different visual encodings (e.g., colors), making the structural composition of the board easier to interpret.

2.5.3 Reproducibility and Custom Instances

Instance generation is fully configuration-driven, enabling the exact reproduction of board layouts across experimental runs. By decoupling geometric generation from cost-matrix computation, the same geometric instance can be evaluated under different kinematic assumptions without regeneration.

This design supports controlled experimentation and ensures that performance comparisons across distance metrics and motion models are fair and repeatable.

Finally, the use of a human-readable **JSON** format further extends this design by supporting the definition of custom instances. In addition to automatically generated layouts, board configurations can be manually edited or specified directly by the user, allowing the precise encoding of ad-hoc or physically observed boards. This makes it possible to bypass stochastic generation entirely when required, while still leveraging the same preprocessing and optimization pipeline.

Overall, this approach provides a high degree of flexibility: instances can be randomly generated, systematically modified, or explicitly handcrafted, all while preserving reproducibility and consistency across experimental evaluations.

This chapter describes the implementation of the Single-Commodity Flow (SCF) formulation introduced previously, using the CPLEX C APIs. The goal is to translate the mathematical model into an executable optimization program capable of solving the generated drilling instances.

The focus is on modeling choices, variable construction and constraint encoding. Low-level details of the CPLEX API are omitted where they do not contribute to understanding the model implementation, but they are still available for inspection in the codebase.

3.1 Input Interface and Data Preparation

The solver operates on a two-stage input pipeline, as explained previously. In the first stage, the instance generator produces the board layout and stores the hole coordinates in a **JSON** file. In the second stage, a dedicated processor computes the travel-time cost matrix based on the selected distance metric and motion parameters, producing a standalone **.dat** file. The solver reads this **.dat** file as input, which begins with a single integer specifying the number of holes, followed by rows of floating-point values representing the travel-time costs between each ordered pair of holes. This design ensures that the optimization algorithm interacts only with the numeric cost matrix, with no interest about distance metrics or motion parameters.

The format of the traditional **.dat** file is shown in the example below:

instance.dat					
5					
0	1	9	9	9	
9	0	1	9	9	
9	9	0	1	9	
9	9	9	0	1	
1	9	9	9	0	

This is naturally a trivial example, but it effectively illustrates the structure of the **.dat** file. From the way the cost matrix is populated, the optimal solution can be immediately identified by inspection. Assuming holes are numbered from 0 (the starting hole) and follow the order in the cost matrix, the resulting trivial sequence of drilling operations is:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0.$$

When the **.dat** file is read, the travel-time values are stored internally in a one-dimensional array rather than a full two-dimensional matrix. This choice improves memory efficiency and access speed for large instances. To maintain clarity and usability, the implementation provides access routines that emulate standard matrix indexing. After this phase, the solver will only need to use the stored values of the cost matrix to proceed in the solve.

3.2 SCF Solver Implementation

The Single-Commodity Flow (SCF) model presented previously is implemented using the CPLEX C APIs, which require encoding the problem in a specific linear form. Constraints are expressed as linear equations or inequalities, and the variables must be properly aligned to their coefficients for the model to be correctly encoded. Additionally, for efficiency reasons (especially when handling large instances), CPLEX requires that the specification of the constraints is provided in a compressed sparse row (CSR) format.

With this framework in place, we now proceed to the implementation of the SCF model, beginning with the definition of decision variables and the objective function. The code snippets provided below are partial and do not include the declaration of all variables or supporting functions; however, their purpose and behavior can be inferred from the names and context. The focus is on the algorithmic steps required to translate the mathematical model into executable CPLEX calls, providing a clear mapping between the formal formulation and its computational implementation. All auxiliary details are omitted for brevity, clarity, and readability.

3.2.1 Decision Variables and Objective Function

The SCF formulation relies on two families of decision variables: binary routing variables y_{ij} and continuous flow variables x_{ij} . In the CPLEX implementation, variables are added incrementally to the model using column-wise construction, which also allows the objective function to be defined at the same time.

The binary variables $y_{ij} \in \{0, 1\}$ indicate whether the drill head moves directly from hole i to hole j in the selected tour and they are the only variables appearing in the objective function. Their cost coefficients are given by the precomputed travel-time matrix.

```
1 double lb = 0.0, ub = 1.0, y_cost = 0; char y_type = 'B';
2 for (size_t i = 0; i < n; ++i) {
3     for (size_t j = 0; j < n; ++j) {
4         if (j == i) continue; /* skip self-loops */
5
6         y_cost = m_instance.cost(i, j);
7         CPXnewcols(
8             m_cpx_env, /* CPLEX environment (env) */
9             m_cpx_lp, /* CPLEX problem (lp) */
10            1, /* num vars created (ccnt) */
11            &y_cost, /* obj coefficients (obj) */
12            &lb, &ub, /* var bounds (lb, ub) */
13            &y_type, /* var type (xctype) */
14            &y_name /* var name (colname) */
15        );
16        m_ymap[i * n + j] = m_cpx_curr_var_pos++; /* for referencing y_ij later */
17    }
18 }
```

Listing 1: Partial¹ implementation for creating y_{ij} variables in the model.

The snippet provided in Listing 1 illustrates the creation of the y_{ij} variables, one at a time. Each variable is defined with lower and upper bounds equal to 0 and 1, respectively, and is declared as binary. Variables corresponding to self-loops ($i = j$) are explicitly excluded (i.e. not created), as transitions from a node to itself are meaningless in the routing context and do not appear in the mathematical model. Starting from the full $n \times n$ grid of possible ordered pairs, this exclusion removes exactly n self-loop variables (the diagonal of matrix whose entries are y_{ij} , which corresponds to the diagonal of the cost matrix of any generic instance). The result is a lower number of variables, that is, $n^2 - n$ instead of n^2 .

Finally, the index of each newly created variable is stored in a lookup structure, which allows the implementation to later recover the column index associated with a specific pair (i, j) when assembling constraints and extracting the values of the variables in the solution.

The continuous variables $x_{ij} \geq 0$ represent the amount of flow sent along arc (i, j) and are used exclusively to enforce connectivity and eliminate subtours. Unlike the routing variables, they do not contribute to the objective function. The following snippet shows the creation of the x_{ij} variables, one at a time.

```

1  double lb = 0.0, ub = CPX_INFBOUND; char x_type = 'C';
2  for (size_t i = 0; i < n; ++i) {
3      for (size_t j = 0; j < n; ++j) {
4          if (j == i || j == 0)                /* skip self-loops and flow */
5              continue;                        /* directed towards node 0 */
6
7          CPXXnewcols(
8              m_cpx_env,                        /* CPLEX environment (env) */
9              m_cpx_lp,                        /* CPLEX problem (lp) */
10             1,                               /* num vars created (ccnt) */
11             NULL,                            /* obj coefficients (obj) */
12             &lb, &ub,                        /* var bounds (lb, ub) */
13             &x_type,                         /* var type (xctype) */
14             &x_name                           /* var name (colname) */
15         );
16         m_xmap[i * n + j] = m_cpx_curr_var_pos++; /* for referencing x_ij later */
17     }
18 }

```

Listing 2: Partial implementation for creating x_{ij} variables in the model.

These variables are declared as continuous, with a lower bound of zero and an unbounded upper limit. Two types of arcs are excluded during creation: self-loops, as before, are skipped and, in addition, variables corresponding to flow directed towards node 0 are omitted. This reflects the SCF convention adopted in our model, where node 0 acts as the root of the flow and does not receive incoming flow. Excluding these variables reduces the size of the model without altering its correctness. More specifically, from the initial n^2 candidate

¹Some things are omitted for brevity and clarity, but their meaning can be inferred from the naming or the context of the snippet. The full implementation remains available in the codebase.

flow variables, this exclusion effectively eliminates $n + n - 1$ variables from the model. Consequently, the number of flow variables reduces from n^2 to $(n - 1)(n - 1) = (n - 1)^2$.

Since the flow variables do not appear in the objective function, the objective coefficients passed to the function call are set to `NULL`, which implicitly assigns them zero cost. As with the routing variables, each x_{ij} is recorded in a dedicated index map to allow consistent access when defining the flow conservation and linking constraints.

Both variable families are added using the same column-wise mechanism, highlighting how CPLEX treats all decision variables uniformly, regardless of their role in the model. Overall, the decision vector comprises $n(n - 1)$ binary routing variables and $(n - 1)^2$ continuous flow variables, for a total of $2n^2 - 3n + 1 < 2n^2$ decision variables.

The explicit bookkeeping of variable indices is a recurring pattern throughout the implementation and is crucial for bridging the gap between the mathematical formulation, which is naturally indexed by node pairs, and the solver interface, which operates on linear arrays. This design choice keeps the constraint-generation logic readable and closely aligned with the SCF model presented earlier.

3.2.2 Constraints

Before introducing the individual constraints, it is useful to clarify how CPLEX represents linear models internally and how the mathematical formulation is mapped to the solver interface. In particular, two aspects are crucial for understanding the implementation: the linear algebraic form used to encode constraints, and the sparse representation adopted for the constraint matrix.

As mentioned previously, CPLEX requires all linear constraints to be expressed in the form

$$\mathbf{A}\hat{\mathbf{x}} \sim \mathbf{b}, \quad \sim \in \{\leq, =, \geq\}, \quad (3.6)$$

where $\hat{\mathbf{x}}$ is the column vector of *all* decision variables, $\mathbf{A}$ is the constraint coefficient matrix, and $\mathbf{b}$ is the RHS column vector. From the solver's perspective, there is no notion of multidimensional indexing: all decision variables are stored in a single linear array, in the exact order in which they are created. In our implementation, this order reflects the sequence in which the y_{ij} binary variables are added first, followed by the x_{ij} continuous variables. At the modeling level, this can be interpreted as a partition of the decision vector,

$$\hat{\mathbf{x}} = \begin{bmatrix} \mathbf{y} \\ \mathbf{x} \end{bmatrix} = \left[\underbrace{y_{0,1} \cdots y_{p,\ell} \cdots y_{n-1,n-1}}_{\substack{n(n-1) \text{ binary} \\ \text{routing variables}}} \mid \underbrace{x_{0,1} \cdots x_{q,r} \cdots x_{n-1,n-1}}_{\substack{(n-1)^2 \text{ continuous} \\ \text{flow variables}}} \right]^T, \quad (3.7)$$

where $p, q \in \{0, 1, \dots, n - 1\}$, $\ell \in \{0, 1, \dots, n - 1\}$, $\ell \neq p$ and $r \in \{1, 2, \dots, n - 1\}$, $r \neq q$. As a consequence, the coefficient matrix $\mathbf{A}$ is implicitly structured according to this ordering, even though this structure is not visible to the solver itself. Each constraint row then selects and weights specific entries of this vector $\hat{\mathbf{x}}$. To make this concept more concrete, we now consider the illustrative generic example, presented below, in which the column vector is

explicitly partitioned into routing variables $\mathbf{y}$ and flow variables $\mathbf{x}$. The resulting matrix representation highlights how coefficients associated with different variable families occupy distinct regions of the matrix, even though they are treated uniformly by the solver.

$$\begin{array}{c}
\hat{x} \text{ coefficients} \\
\hline
\begin{array}{cc}
\mathbf{y} \text{ coefficients} & \mathbf{x} \text{ coefficients} \\
\hline
\begin{array}{ccccccc}
\lambda_{0,1}^{(0)} & \cdots & \lambda_{p,\ell}^{(0)} & \cdots & \lambda_{n-1,n-1}^{(0)} & \mu_{0,1}^{(0)} & \cdots & \mu_{q,r}^{(0)} & \cdots & \mu_{n-1,n-1}^{(0)} \\
\vdots & & & & & & & & & \\
\lambda_{0,1}^{(c)} & \cdots & \lambda_{p,\ell}^{(c)} & \cdots & \lambda_{n-1,n-1}^{(c)} & \mu_{0,1}^{(c)} & \cdots & \mu_{q,r}^{(c)} & \cdots & \mu_{n-1,n-1}^{(c)} \\
\vdots & & & & & & & & & \\
\lambda_{0,1}^{(m-1)} & \cdots & \lambda_{p,\ell}^{(m-1)} & \cdots & \lambda_{n-1,n-1}^{(m-1)} & \mu_{0,1}^{(m-1)} & \cdots & \mu_{q,r}^{(m-1)} & \cdots & \mu_{n-1,n-1}^{(m-1)}
\end{array} \\
\hline
\begin{array}{cc}
n(n-1) = n^2 - n & (n-1)^2
\end{array}
\end{array}
\end{array}
\left[\begin{array}{c}
y_{0,1} \\
\vdots \\
y_{p,\ell} \\
\vdots \\
y_{n-1,n-1} \\
\hline
x_{0,1} \\
\vdots \\
x_{q,r} \\
\vdots \\
x_{n-1,n-1}
\end{array} \right] \sim \left[\begin{array}{c}
b_0 \\
\vdots \\
b_c \\
\vdots \\
b_{m-1}
\end{array} \right]
\end{array}
\left. \vphantom{\begin{array}{c} \hat{x} \text{ coefficients} \end{array}} \right\} \mathbf{b}$$

After introducing this example, the generic constraint indexed by $0 \leq c < m$ can be interpreted by isolating the c -th row of $\mathbf{A}$. The actual constraint enforces that the inner product of this row with the decision vector $\hat{\mathbf{x}}$ equals the corresponding value in the RHS, given by the c -th entry of $\mathbf{b}$. In mathematical terms, this generic constraint can be written as

$$\sum_{p=0}^{n-1} \sum_{\substack{\ell=0 \\ \ell \neq p}}^{n-1} \lambda_{p,\ell}^{(c)} y_{p,\ell} + \sum_{q=0}^{n-1} \sum_{\substack{r=1 \\ r \neq q}}^{n-1} \mu_{q,r}^{(c)} x_{q,r} \sim b_c, \quad (3.8)$$

where the coefficients $\lambda_{p,\ell}^{(c)}$ and $\mu_{q,r}^{(c)}$ are nonzero only for the variables participating in that specific constraint. In practice, each constraint involves only a small subset of the decision variables. As a consequence, most coefficients in any given row of the matrix $\mathbf{A}$ are equal to zero, and the matrix itself is highly sparse. For this reason, explicitly storing or manipulating all zero coefficients would be both wasteful and unnecessary. Indeed, CPLEX does not require the coefficient matrix to be provided in dense form. Instead, constraints are encoded using a compressed sparse row (CSR) representation, which stores only the nonzero coefficients of each row together with their associated column indices. A separate row pointer array records the start of each row, allowing the solver to consider all constraints in a very compact structure.

With these concepts in place, we are now ready to translate the abstract SCF constraints into their concrete implementation within CPLEX.

The SCF formulation requires four families of constraints to properly model the routing problem. These constraints enforce both the connectivity of the solution and the correct interaction between the routing and flow variables:

1. Flow conservation constraints, which ensure that the flow originating from the root node is correctly propagated through the tour.
2. In-degree constraints, which guarantee that each node is entered exactly once (i.e. from a single arc of its incoming arcs).
3. Out-degree constraints, which guarantee that each node is exited exactly once (i.e. through a single arc of its outgoing arcs).
4. Linking constraints, which tie the routing variables y_{ij} to the flow variables x_{ij} , ensuring consistency between the decision to traverse an arc and the flow passing through it.

We will now examine the implementation of each constraint type, highlighting how the abstract mathematical definitions are mapped into CPLEX API calls.

Flow Conservation Constraints. The implementation below illustrates how these constraints are encoded in CPLEX, focusing on the core logic while omitting boilerplate for clarity.

```

1  double rhs = 1.0; char sense = 'E';
2  CPXNNZ rmatbeg = 0;
3  for (size_t k = 1; k < n; ++k) {                               /* note we start from 1      */
4      rmatind.clear(); rmatval.clear();                           /* clear for new constraint */
5
6      /* first sum                                              */
7      for (size_t i = 0; i < n; ++i) {
8          idx_x_ik = m_xmap[i * n + k];
9          if (idx_x_ik == -1) continue;                          /* only consider existing vars */
10
11         rmatind.push_back(idx_x_ik);
12         rmatval.push_back(1.0);
13     }
14
15     /* second sum (recall we have negative coefficients here) */
16     for (size_t j = 1; j < n; ++j) {                            /* note we start from 1      */
17         idx_x_kj = m_xmap[k * n + j];
18         if (idx_x_kj == -1) continue;                          /* only consider existing vars */
19
20         rmatind.push_back(idx_x_kj);
21         rmatval.push_back(-1.0);
22     }
23
24     CPXaddrows(/* ... see next snippet ... */)                  /* constraint creation      */
25 }

```

Listing 3: Partial implementation for creating flow conservation constraints (1/2).

The previous snippet shows the high-level function call that adds a single flow conservation constraint. We consider the detailed function call with all the required parameters in the snippet presented below. The implementation creates one constraint at a time (i.e. one for every function call).

```

1  CPXxaddrows(
2      m_cpx_env,                /* CPLEX environment      (env)      */
3      m_cpx_lp,                 /* CPLEX problem          (lp)       */
4      0,                        /* num vars created        (ccnt)   */
5      1,                        /* num cons created        (rcnt)   */
6      rmatind.size(),           /* cons num vars involved  (nzcnt)  */
7      &rhs,                     /* cons right hand side    (rhs)    */
8      &sense,                   /* cons sense              (sense)   */
9      &rmatbeg,                 /* cons var matbeg         (rmatbeg) */
10     rmatind.data(),            /* cons var indices        (rmatind) */
11     rmatval.data(),            /* cons var coefficients    (rmatval) */
12     NULL,                     /* var name                (colname) */
13     &row_name                  /* cons name               (rowname) */
14 );

```

Listing 4: Partial implementation for creating flow conservation constraints (2/2).

Now that we have the full overview the implementation, we can proceed with a brief explanation that maps the code to the mathematical formulation of the flow constraints previously discussed in (1.3). Recall the constraint definition:

$$\sum_{i:(i,k) \in \mathcal{A}} x_{ik} - \sum_{j:(k,j) \in \mathcal{A}} x_{kj} = 1 \quad \forall k \in \mathcal{N} \setminus \{0\}.$$

Since the flow conservation constraint has the same structure for every node, we focus on a single, fixed $k \in \mathcal{N} \setminus \{0\}$ to illustrate the logic. All other nodes are handled analogously within the loop beginning at line 3 of Listing 3.

The first summation term is implemented by the loop starting at line 7 of Listing 3. For the fixed node k , the loop iterates over all indices i such that the arc $(i, j) \in \mathcal{A}$ corresponds to a valid flow variable in the model. The existence of the variables is checked via the lookup map. For each valid x_{ik} , the column index is added to the constraint structure with a coefficient of $+1$, thus representing the incoming-flow term

$$\sum_{i:(i,k) \in \mathcal{A}} x_{ik}.$$

The second summation term is implemented by the loop starting at line 16 of Listing 3. For the fixed node k , the loop iterates over all indices j such that the arc $(k, j) \in \mathcal{A}$ corresponds to a valid flow variable in the model. The loop starts from $j = 1$, reflecting the fact that variables representing flow directed toward the root node 0 were excluded during variable creation. As before, the existence of the variables is verified using the lookup map, and only valid variables are included in the constraint. For each such variable, the corresponding column index is added with a coefficient of -1 , thereby encoding the outgoing-flow term

$$- \sum_{j:(k,j) \in \mathcal{A}} x_{kj}.$$

Once all nonzero coefficients and their corresponding variable indices have been collected, the constraint is added to the model through a call to `CPXXaddrows` with the appropriate parameters. The sense is set to equality, and the RHS is set to 1.

In-degree and Out-degree constraints. We now turn to the in-degree and out-degree constraints, which jointly enforce the routing structure of the solution. Both constraint families share a similar structure and implementation pattern, differing only in the direction of the arcs involved. For this reason, they are presented together, highlighting the common logic while pointing out the minor differences where relevant.

Recall the in-degree and out-degree constraints presented in (1.4), which enforce that each node is entered exactly once and exited exactly once, respectively:

$$\sum_{i:(i,j) \in \mathcal{A}} y_{ij} = 1 \quad \forall j \in \mathcal{N}, \quad \sum_{j:(i,j) \in \mathcal{A}} y_{ij} = 1 \quad \forall i \in \mathcal{N}.$$

The snippets below provide the implementation of these constraints.

```

1  double rhs = 1.0; char sense = 'E';
2  CPXNNZ rmatbeg = 0;
3  for (size_t j = 0; j < n; ++j) {
4      rmatind.clear(); rmatval.clear();          /* clear for new constraint */
5
6      for (size_t i = 0; i < n; ++i) {
7          idx_y_ij = m_ymap[i * n + j];
8          if (idx_y_ij == -1) continue;          /* only consider existing vars */
9
10         rmatind.push_back(idx_y_ij);
11         rmatval.push_back(1.0);
12     }
13
14     CPXXaddrows(
15         m_cpx_env,          /* CPLEX environment (env) */
16         m_cpx_lp,           /* CPLEX problem (lp) */
17         0,                  /* num vars created (ccnt) */
18         1,                  /* num cons created (rcnt) */
19         rmatind.size(),     /* cons num vars involved (nzcnt) */
20         &rhs,               /* cons right hand side (rhs) */
21         &sense,             /* cons sense (sense) */
22         &rmatbeg,           /* cons var matbeg (rmatbeg) */
23         rmatind.data(),     /* cons var indices (rmatind) */
24         rmatval.data(),     /* cons var coefficients (rmatval) */
25         NULL,              /* var name (colname) */
26         &row_name          /* cons name (rowname) */
27     );
28 }
```

Listing 5: Partial implementation for creating in-degree constraints.

The outer loop iterates over all nodes $j \in \mathcal{N}$, generating one constraint per node. For each fixed j , the inner loop scans all possible i and checks whether the routing variable y_{ij} exists in the model. As before, this verification is performed using the lookup map, which filters out self-loops and any arc that was excluded during variable creation.

For every valid arc (i, j) , the corresponding variable index is added to the sparse row representation with coefficient $+1$, thereby constructing the left-hand side of the constraint. The right-hand side is set to 1, and the constraint sense is equality, ensuring that exactly one incoming arc is selected for node j .

Once all relevant coefficients have been collected, the constraint row is added to the model via a call to `CPXXaddrows`. This process is repeated for each node, yielding the full family of in-degree constraints.

This pattern will be reused with minimal modifications for the out-degree constraints, where the roles of incoming and outgoing arcs are exchanged.

```

1  double rhs = 1.0; char sense = 'E';
2  CPXNNZ rmatbeg = 0;
3  for (size_t i = 0; i < n; ++i) {
4      rmatind.clear(); rmatval.clear();          /* clear for new constraint */
5
6      for (size_t j = 0; j < n; ++j) {
7          idx_y_ij = m_ymap[i * n + j];
8          if (idx_y_ij == -1) continue;          /* only consider existing vars */
9
10         rmatind.push_back(idx_y_ij);
11         rmatval.push_back(1.0);
12     }
13
14     CPXXaddrows(
15         m_cpx_env,          /* CPLEX environment      (env) */
16         m_cpx_lp,           /* CPLEX problem          (lp) */
17         0,                  /* num vars created        (ccnt) */
18         1,                  /* num cons created        (rcnt) */
19         rmatind.size(),      /* cons num vars involved  (nzcnt) */
20         &rhs,               /* cons right hand side    (rhs) */
21         &sense,             /* cons sense              (sense) */
22         &rmatbeg,           /* cons var matbeg         (rmatbeg) */
23         rmatind.data(),      /* cons var indices        (rmatind) */
24         rmatval.data(),      /* cons var coefficients    (rmatval) */
25         NULL,               /* var name                 (colname) */
26         &row_name           /* cons name                (rowname) */
27     );
28 }
```

Listing 6: Partial implementation for creating out-degree constraints.

For each node $i \in \mathcal{N}$, the corresponding constraint enforces that exactly one outgoing arc (i, j) is selected, ensuring that every node is exited exactly once.

Linking Constraints. The final family of constraints consists of the linking constraints, which connect the routing variables with the flow variables. In the SCF formulation, these constraints take the form presented in (1.5), that can be arranged as

$$x_{ij} + (1 - n)y_{ij} \leq 0 \quad \forall (i, j) \in \mathcal{A}, \quad j \neq 0,$$

to ensure all variables appear on the left-hand side. The constant $n - 1$ represents a valid upper bound on the amount of flow that can traverse any arc and serves as the big- M coefficient. Without these constraints, the flow variables could route flow through arcs that are not part of the selected tour, breaking the logical connection between routing and connectivity.

The implementation closely follows the patterns established in the previous sections, and is presented in the snippet below.

```

1  double rhs = 0.0; char sense = 'L';
2  CPXNNZ rmatbeg = 0;
3  for (size_t i = 0; i < n; ++i) {
4      for (size_t j = 0; j < n; ++j) {
5          idx_x_ij = m_xmap[i * n + j];
6          idx_y_ij = m_ymap[i * n + j];
7
8          if (idx_x_ij == -1) continue;          /* only consider existing vars */
9
10         rmatind[0] = idx_x_ij;
11         rmatind[1] = idx_y_ij;
12
13         rmatval[0] = 1.00;
14         rmatval[1] = 1.0 - n;
15
16         CPXXaddrows(
17             m_cpx_env,          /* CPLEX environment      (env) */
18             m_cpx_lp,          /* CPLEX problem          (lp) */
19             0,                  /* num vars created       (ccnt) */
20             1,                  /* num cons created       (rcnt) */
21             rmatind.size(),     /* cons num vars involved (nzcnt) */
22             &rhs,               /* cons right hand side   (rhs) */
23             &sense,             /* cons sense             (sense) */
24             &rmatbeg,           /* cons var matbeg        (rmatbeg) */
25             rmatind.data(),     /* cons var indices       (rmatind) */
26             rmatval.data(),     /* cons var coefficients   (rmatval) */
27             NULL,               /* var name               (colname) */
28             &row_name          /* cons name              (rowname) */
29         );
30     }
31 }
```

Listing 7: Partial implementation for creating linking constraints.

A key difference from the previous constraints is that each linking constraint involves exactly two variables: one flow variable and its corresponding routing variable. There is no summation over indices here, no accumulation across rows or columns. Each arc produces one constraint, and each constraint touches precisely two columns of the coefficient matrix. The right-hand side is set to zero and the constraint sense is 'L', corresponding to a “less-than-or-equal” inequality. One last remark clarifies the role of the nested loops: they repeat this constraint for every admissible arc $(i, j) \in \mathcal{A}$. Each iteration of the loops corresponds to one distinct linking constraint and therefore to one distinct row added to the model.

To summarize it all, the degree constraints shape the tour, the flow conservation constraints enforce connectivity, and the linking constraints glue the two worlds together. With all of them in place, the SCF formulation is complete, and the implementation now matches the mathematical model.

3.3 Solution Extraction and Interpretation

Having defined and solved the model, we now address the final step of the pipeline: extracting and interpreting the solution produced by CPLEX. Although the solver returns values for all decision variables, only the routing variables y_{ij} are needed to reconstruct the drilling order.

CPLEX provides the solution as a flat array of variable values, indexed according to the order in which variables were added to the model. The implementation must therefore identify the entries corresponding to the routing variables, map them back to their (i, j) indices, and assemble a valid tour. The following snippet outlines the procedure to do so.

```

1  size_t current = 0; std::vector<int> sequence; sequence.push_back(0);
2
3  while (true) {
4      bool found = false;
5
6      for (size_t j = 0; j < n && !found; ++j) {
7          const idx y_idx = m_ymap[current * n + j];
8          if (y_idx == -1) continue;           /* only consider existing vars */
9
10         if (all_vars[y_idx] > 1e-6) {
11             current = j;
12             sequence.push_back(j);
13             found = true;
14         }
15     }
16
17     if (current == 0)
18         break;
19 }
20
21 return sequence;

```

Listing 8: Partial implementation for reconstructing the hole sequence from the variables.

The reconstruction procedure exploits a simple property of the SCF formulation: in a feasible solution, each node has exactly one outgoing routing arc selected. As a consequence, the active y_{ij} variables define a single directed cycle, which can be recovered by starting from the source node and repeatedly following the selected outgoing arc until the tour closes.

The code begins by initializing the reconstruction at node 0, which plays the role of the reference starting point in the model. The vector `sequence` is used to store the ordered list of visited nodes, and it is seeded with the starting node. At each iteration of the outer loop, the algorithm scans all potential successors of the current node by using the lookup map to translate a logical index pair into the corresponding index position in the linear array that stores all the variables. When a routing variable with value greater than a small numerical tolerance is found, the corresponding arc is interpreted as selected in the solution. The algorithm then advances to the node just found, appends it to the sequence, and continues the process from there in a new iteration. Since the model enforces exactly one outgoing arc per node, the first such variable encountered is guaranteed to be the correct successor. The outer loop terminates once the traversal returns to node 0, indicating that the cycle has been closed and the full tour has been reconstructed.

Having the solution as an explicit sequence of hole indices is extremely practical: it provides a direct, human-readable representation of the tour, rather than a raw collection of variable values. This format makes it trivial to visualize the path on the board, highlighting the actual drilling order and enabling immediate inspection of patterns or inefficiencies. Moreover, it facilitates quantitative comparisons: heuristic solutions can be directly aligned with the optimal sequence, allowing for easy computation of deviations, travel-time differences, or other performance metrics.

By converting the solver’s output into this structured form, we bridge the gap between the abstract mathematical solution and the real-world object associated with an instance.

3.4 Running the SCF Solver

Now that the model has been fully implemented, we can turn to the actual execution of the solver. At this stage, the `C++` code acts as a self-contained program that reads a preprocessed instance (the cost-matrix `.dat` file), builds the SCF model in CPLEX, and invokes the optimization routine to find an optimal routing.

For compilation and execution, we rely on a `Makefile`-based workflow. This ensures that all source files, including the interface with the CPLEX C APIs, are compiled with the correct flags, libraries, and include paths.

3.4.1 Compilation

Before running the solver, the system must be properly compiled. The code relies on a `C++` compiler supporting at least `C++11`, such as `g++`, and requires access to the CPLEX C APIs headers and libraries. Compilation is handled through the provided `Makefile`. When executed, the `Makefile` organizes the build process by first creating a `build/` directory where

all intermediate object files are stored, and then producing the final executables in the `bin/` directory. This structure keeps source and binary files separate and allows for a clean rebuild.

To compile the code, you can simply run the following commands.

```
$ make clean    # safely removes old object files and binaries for a fresh rebuild
$ make all
```

If everything compiles successfully, the `bin/` folder will contain two executables. The first, `single_instance_cplex_solver`, is designed to solve a single `.dat` instance at a time, allowing for detailed inspection of the solution and intermediate outputs. The second, `batch_cplex_solver`, is intended to process multiple instances sequentially, producing results suitable for aggregate analysis and performance benchmarking across a dataset.

With the compilation complete and the executables in place, we can now proceed to demonstrate their use.

3.4.2 Solving an Instance

To solve a single instance, the `single_instance_cplex_solver` executable is employed. The full usage is the following.

```
$ ./bin/single_instance_cplex_solver <filename> [--verbose <0|1|2>] [--average <n_runs>]
```

The program expects the path to an input `.dat` file as a mandatory argument, which contains the number of nodes and the cost matrix for the instance. Two optional flags can also be provided: the first is `--verbose`, which controls the level of console output and defaults to `1` if omitted, and the second is `--average`, which specifies the number of independent solver runs over which the results are averaged, with a default value of `5`. Averaging multiple runs is particularly useful for obtaining robust performance metrics and mitigating variability due to solver heuristics or numerical factors.

To take a concrete example, suppose we have an instance `example.dat` for which we want the solution and the average solve time that results from the execution. Then we could run the command that follows.

```
$ ./bin/single_instance_cplex_solver example.dat --verbose 1 --average 10
```

Here we have required that the results come from averaging `10` runs of the solver on the same instance. We can see that the solver takes the `.dat` file as input, which confirms that it operates solely on the precomputed cost matrix. It does not have access to the original board geometry, distance metrics, or kinematic parameters. This separation ensures that the solver remains fully agnostic to how the instance was created, allowing the same executable to handle any valid cost matrix without modification.

If needed, the output of the solver can be redirected to a file using standard shell redirection. This is particularly useful when running experiments or collecting results for

post-processing and plotting. For instance, appending `> results.dat` to the command will store all console output in a file instead of printing it to standard output.

3.4.3 Batch Solver

The batch solver provides an automated way to solve multiple instances in a single execution. Instead of operating on a single `.dat` file, it takes as input a directory containing a collection of instances and processes them sequentially using the same SCF model and solver configuration. The command syntax is:

```
$ ./bin/batch_cplex_solver <instances_folder> [--average <n_runs>] [--output <filename>]
```

The only mandatory argument is `instances_folder`, which must point to a directory containing the preprocessed `.dat` files to be solved. The optional `--average` parameter specifies the number of repeated runs performed for each instance, allowing solution times and objective values to be averaged for more robust performance evaluation. If omitted, a default value of 5 is used, consistent with the single-instance solver. The optional `--output` filename parameter controls where the aggregated results are written; if this option is not provided, all results are saved by default to a file named `results.dat` in the working directory.

As a concrete example, assuming the instances are stored in a dedicated directory named `data/instances/`, the batch solver can be invoked as follows:

```
$ ./bin/batch_cplex_solver data/instances/ --average 10 --output batch_results.dat
```

This command processes all `.dat` files found in the `data/instances/` directory, runs the solver ten times per instance to compute averaged results, and writes the aggregated output to `batch_results.dat`.

Finally, all commands and paths shown above (for both solvers) assume execution from the project's root directory (i.e., where the `Makefile` is located) and must be adapted accordingly if run from a different location.

3.5 Sanity Check for Model Correctness

Before tackling full-scale instances, it is useful to verify that the SCF model and its implementation behave as expected on a tiny, trivial example. For instance, we can create a board with only a few holes and a cost matrix designed so that the optimal tour is immediately apparent by inspection. Running the solver on this instance ensures that the variables are correctly defined, constraints are properly enforced, and the solution extraction logic reconstructs the expected sequence. This simple check provides confidence that the model implementation is correct and ready for more complex, realistic problems.

The instance used for this sanity check is deliberately trivial and the corresponding file is `sanity_check.dat`. It contains a 5×5 cost matrix in which the costs are set such that only one path yields the minimal total cost, while all alternative paths are prohibitively expensive.

Figure 4 shows the cost matrix, alongside the output of `single_instance_cplex_solver` when executed on this instance with the `--average` parameter set to 10.

Running the solver

```
$ ./bin/single_instance_cplex_solver data/instances/sanity_check.dat --average 10
```

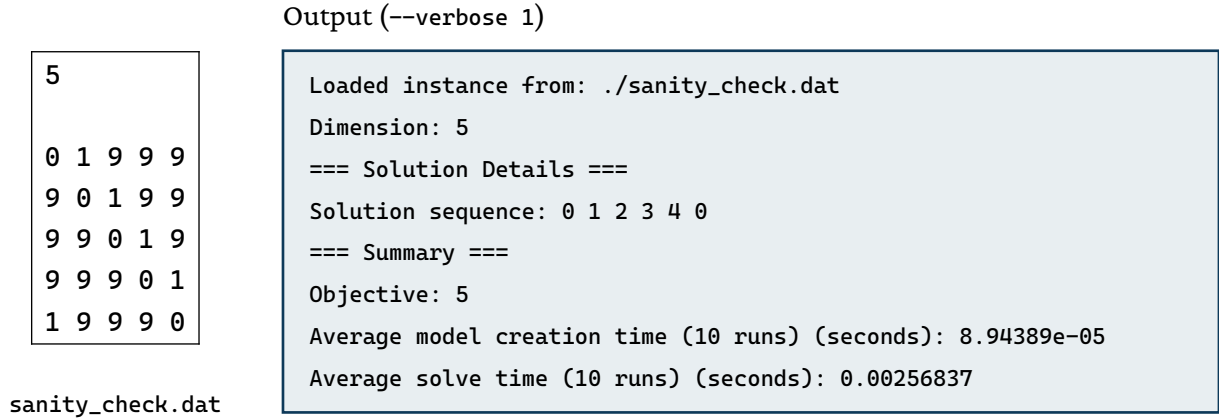


Figure 4: Example run of the solver on a trivial instance to check model correctness.

The output shows that the solver correctly identifies the optimal tour

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$$

with an objective value of 5. The reported average model creation time is negligible (8.94×10^{-5} seconds), and the average solve time is 0.00257 seconds, demonstrating that the solver efficiently handles small instances while producing the expected optimal solution.

It is worth emphasizing that, while this instance is trivial and immediately solvable by inspection, the cost values may not correspond to realistic travel times. Interpreted as literal distances between holes on a physical board (by forgetting for a moment that we use travel times), the numbers would be implausible. However, they can be rationalized in a hypothetical “pathological” scenario where the drilling head is strongly constrained: the low-cost transitions represent the only convenient moves, and any deviation carries a prohibitive penalty.

Finally, this is yet again another confirmation of a fact that we have seen over and over at this point: the solver treats the data abstractly. It only sees decision variables, constraints, and numerical coefficients, and computes an optimal solution accordingly, regardless of whether these numbers represent distances, travel times, or arbitrary penalties. This confirms that the model is fully agnostic to the underlying semantics of the instance, and that it just looks at the numbers.

With the SCF model implemented, tested, and a fair amount of sleep deprivation behind us (hoping none of this has been too boring) we can now move on to a slightly more entertaining part: implementing a heuristic algorithm for our problem and discussing a detailed comparison with the exact method.

This chapter introduces a heuristic approach to the drilling optimization problem, focusing on a two-opt tabu search metaheuristic. After presenting its core principles and implementation, we discuss how the specific structure of the problem can be exploited to improve both efficiency and solution quality. These observations motivate the design and implementation of enhanced variants of the basic method, leading to a more effective metaheuristic tailored to the characteristics of the drilling path optimization problem.

A brief disclaimer is in order. The final algorithm presented in this chapter is the result of many hours of development, including numerous failed attempts and iterative refinements. Over time, the method evolved into a sufficiently complex system that a fully detailed description of every design choice and optimization is no longer feasible within the scope of this report. Many small implementation details contribute to its overall efficiency, but addressing all of them would require disproportionate space. For completeness and transparency, the full codebase remains available for inspection and documents all algorithmic details and implementation choices.

4.1 Local Search Algorithm Design

For the heuristic approach, we rely on a local search strategy based on the well-known two-opt move, a standard technique in the context of the Traveling Salesman Problem. Two-opt iteratively improves a candidate tour by removing two edges and reconnecting the resulting paths in the alternative way. Despite its conceptual simplicity, this move is particularly effective at eliminating crossings and reducing overall tour length, making it a natural baseline for problems that involve finding efficient orderings over a set of locations, such as the drilling sequence considered here.

4.1.1 Solution Representation

A solution, that is, a candidate drilling tour, is represented as an ordered sequence of holes specifying the order in which drilling operations are performed. This representation is particularly well suited to two-opt local search, since a two-opt move reduces to reversing a contiguous subsequence of the tour, an operation that can be implemented efficiently.

In the implementation, a solution is encapsulated in a **Tour** object, which is essentially a lightweight wrapper around a `std::vector` storing the sequence of hole indices. The representation is open, meaning that the tour is stored as a simple sequence visiting all nodes exactly once, without explicitly repeating the starting node at the end. This design choice simplifies the handling of two-opt moves, especially those involving the boundaries of the sequence. Importantly, this does not alter the semantics of the solution: when evaluating a tour, the closing edge from the last hole in the sequence back to the first one is always taken into account. In other words, although the tour is represented internally as an open

path, it is conceptually interpreted as a closed cycle, defined by the sequence together with the implicit final edge connecting its endpoints.

4.1.2 Finding Initial Solutions

The initial solution for the local search is constructed using a nearest neighbor heuristic. This choice is motivated by three main considerations. First, even a simple greedy construction based on nearest neighbors produces solutions that are vastly superior to a randomly generated tour, providing a much better starting point for local improvement. Second, nearest neighbor is straightforward to implement: it builds a tour incrementally by repeatedly selecting the closest unvisited hole, making the construction logic simple, efficient, and easy to integrate into the overall framework. Third, and most importantly in the context of a multi-start strategy, nearest neighbor naturally introduces diversity. Because the greedy selection heavily depends on the starting node, initializing the algorithm from different starting holes leads to distinct tours that tend to share locally good edges but differ globally, which is desirable for exploration.

Alternative construction heuristics, such as farthest insertion, were also considered due to their better worst-case approximation properties and greater stability. However, these methods tend to produce more similar initial tours across different runs, reducing diversity. Given the intention to rely on multiple independent starts, we favored a simpler construction heuristic that promotes variability among initial solutions, with the expectation that this increased diversity would compensate for the slightly weaker average quality of the initial tours once local search is applied.

The code in Listing 9 illustrates the core logic of the nearest neighbor construction heuristic used to generate an initial tour. The snippet shows how a feasible solution is built incrementally, starting from a chosen hole and extending the tour one node at a time.

```
1  /* Add the starting node to the tour */
2  size_t curr_idx = start_idx;
3  tour.push_back(nodes[curr_idx]);
4  visited[curr_idx] = 1;
5
6  for (size_t step = 1; step < n_nodes; ++step) { /* iteratively build the tour */
7      curr_idx = find_nearest_unvisited(nodes[curr_idx], nodes, visited, costs);
8      tour.push_back(nodes[curr_idx]);
9      visited[curr_idx] = 1;
10 }
11
12 return tour;
```

Listing 9: Partial implementation for the Nearest Neighbour.

The procedure begins by inserting the selected starting node into the tour and marking it as visited. At each subsequent step, the algorithm identifies, among all unvisited nodes, the one that is closest to the current position according to the precomputed cost matrix. This node is then appended to the tour and marked as visited. The process repeats until all holes

have been included exactly once, resulting in a complete open sequence that represents a valid tour.

4.1.3 Defining the Neighbourhood of a Solution

In the context of our heuristic, the neighborhood of a solution is defined by all tours that can be obtained from the current one through a single two-opt move.

Given a tour represented as an ordered sequence of n holes, a two-opt move selects two distinct indices $i < j$ and reverses the subsequence between them. In terms of the solution representation, this operation removes the two edges (v_{i-1}, v_i) and (v_j, v_{j+1}) and replaces them with (v_{i-1}, v_j) and (v_i, v_{j+1}) , respectively, implicitly reconnecting the tour while preserving feasibility. Because the tour is stored as an open sequence, the reversal is implemented directly on the underlying vector, while the closing edge between the last and first elements is still accounted for during evaluation. Enumerating the full neighborhood therefore amounts to iterating over all admissible index pairs (i, j) with $0 \leq i < j < n$, excluding degenerate cases that would leave the tour unchanged (i.e., $i = 0 \wedge j = n - 1$, which would just reverse the direction of the current tour making no difference in the cost as long as symmetry is assumed).

The size of this neighborhood grows quadratically with the tour length, on the order of $\mathcal{O}(n^2)$, which is manageable for moderate instance sizes but large enough to motivate efficient evaluation strategies and pruning when embedded in an iterative local search such as tabu search.

4.1.4 Evaluating the Quality of a Solution

Evaluating the quality of a candidate solution amounts to computing the total travel time induced by the corresponding tour. Given a tour represented as an ordered sequence of holes, this is done by summing the travel times associated with each consecutive pair in the sequence, plus the closing edge that connects the last hole back to the first one. Under this representation, a full evaluation requires a single pass over the sequence and therefore runs in linear time $\mathcal{O}(n)$, where n is the number of holes in the (open) tour.

However, for the purpose of local search, repeatedly recomputing the full cost would be unnecessarily expensive. By assuming symmetric instances, we can exploit an important structural property of two-opt moves: any two-opt transformation removes exactly two edges from the current tour and replaces them with two new ones. As a consequence, the cost of a neighboring solution can be obtained in constant time by adjusting the current tour cost, subtracting the costs of the removed edges and adding the costs of the newly introduced ones. This reduces the evaluation of a neighbor from $\mathcal{O}(n)$ to $\mathcal{O}(1)$, which is crucial when exploring a neighborhood of quadratic size.

In practice, the algorithm does not even need to compute the absolute cost of a neighbor. Instead, it is sufficient to compute the cost variation, or delta, associated with a given two-opt move. This delta is defined as the difference between the cost of the candidate tour and the cost of the current tour. Negative deltas correspond to improving moves,

with more negative values indicating better improvements. By ranking candidate moves according to their deltas, the heuristic can efficiently identify promising neighbors without ever explicitly reconstructing or fully evaluating the corresponding tours. This incremental evaluation strategy is a key ingredient in achieving good performance for large instances.

The following snippet shows how the cost variation associated with a two-opt move can be computed efficiently by considering only the edges affected by the move.

```

1 size_t prev = (i == 0) ? n_nodes - 1 : i - 1;
2 size_t next = (j == n_nodes - 1) ? 0 : j + 1;
3
4 return costs(tour[i], tour[next]) + costs(tour[prev], tour[j]) /* created edges */
5        - costs(tour[j], tour[next]) - costs(tour[prev], tour[i]); /* broken edges */

```

Listing 10: Partial implementation for the two-opt delta computation.

Since the tour is represented as an open sequence but evaluated as a cycle, the indices immediately preceding i and following j are handled with wrap-around logic. The returned value corresponds to the delta of the move, computed as the cost of the newly added edges minus the cost of the removed ones.

4.1.5 Exploration Strategies: Selecting the Next Solution

Exploring the solution space requires a clear policy for selecting which neighbor to move to at each iteration. In our setting, this choice reduces to how two-opt moves are evaluated and accepted. One option is a First Improvement strategy, which scans the neighborhood and immediately applies the first move that yields a cost reduction, without inspecting the remaining neighbors. An alternative is Best Improvement, where all feasible two-opt moves are evaluated, the associated cost deltas are computed, and the move leading to the largest improvement is selected. While hybrid strategies are possible, we restrict attention to these two classical approaches for clarity.

In practice, we predominantly rely on Best Improvement. Although it has a higher theoretical complexity due to the full neighborhood evaluation, it consistently performed better in our experiments. Moreover, the systematic traversal of neighbors tends to exhibit better cache locality, partially offsetting its higher asymptotic cost, whereas First Improvement may terminate early but does so in a less predictable memory-access pattern. From an implementation perspective, the two strategies are interchangeable: all routines responsible for neighbor selection are parameterized by a simple policy flag, allowing the exploration strategy to be switched with minimal code changes.

Starting from the initial solution constructed by the nearest-neighbor heuristic, the algorithm iteratively explores the neighborhood of the current tour using two-opt moves. At each iteration, a neighboring solution is selected according to the chosen improvement policy and, if it yields a strictly lower cost, it replaces the current solution. This process is repeated, progressively refining the tour as long as improving moves can be found.

The procedure naturally terminates when a full neighborhood exploration fails to identify any improving two-opt move. At this point, the current tour is locally optimal with respect to the chosen neighborhood structure, meaning that no single two-opt reversal can further reduce the total travel cost. Reaching such a local optimum signals the end of the local search phase, at which point the algorithm either stops or, in more advanced variants, hands control to diversification mechanisms such as restarts or tabu strategies, which we will explore in what follows.

4.2 A First Attempt to Metaheuristics: Tabu Search

To overcome the limitation of local optima, we employ a Tabu Search strategy. This approach allows the algorithm to perform non-improving moves, enabling it to escape local minima. Each move performed is recorded in a tabu list, which temporarily forbids reversing that move for a specified number of iterations, known as the tabu tenure. By controlling which moves are prohibited, the search explores new regions of the solution space while avoiding simple cycles, increasing the chance of finding better solutions beyond local optima.

4.2.1 Setting Up the Tabu List

To efficiently implement the tabu mechanism, we developed a dedicated `TabuList` class that internally represents the tabu list as a hash map. Rather than storing individual nodes, the tabu list keeps track of edges—specifically, the pairs of nodes involved in a move—so that a two-opt swap cannot be immediately undone. This is a stronger restriction than merely tracking nodes, as it directly prevents reversing a particular change to the tour. Each edge (i, j) is treated symmetrically, meaning that (i, j) and (j, i) refer to the same entry in the tabu list. The hash function used to compute the key to use in the map (which is more convenient than using a `std::pair`) yields a unique identifier for each edge: for $i < j$, the hash is computed by placing i in the upper 16 bits, and j in the lower 16 bits of a 32-bit integer (and vice versa if $i > j$).

```
1 inline u32 hash(u16 i, u16 j) const {  
2     return i < j ? ((u32)i << 16) | (u32)j : ((u32)j << 16) | (u32)i; /* bit ops */  
3 }
```

Listing 11: Full implementation for the key computation in the `TabuList`.

When a move is performed, the edge involved is added to the list with a tabu expiration iteration calculated as the current iteration plus a fixed tenure, optionally adjusted by a small oscillating offset to introduce variation in the effective tabu duration. This shift prevents cycles and adds diversity to the search. Checking whether a move is tabu simply queries the hash map for the edge and compares the stored expiration iteration to the current iteration. This behavior is presented in the snippet Listing 12 that follows.

```

1  inline void add_tabu(u16 i, u16 j, size_t curr_iter) {
2      size_t tenure_offset = 0;
3      if (m_tenure_oscillation_range != 0) {
4          tenure_offset = curr_iter % m_tenure_oscillation_range; /* dynamic tenure */
5      }
6
7      m_tabu_list[hash(i, j)] = curr_iter + m_tenure + tenure_offset;
8
9      if (m_tabu_list.size() > 3 * (m_tenure + m_tenure_oscillation_range)) {
10         cleanup(curr_iter);
11     }
12 }
13
14 inline bool is_tabu(u16 i, u16 j, size_t curr_iter) const {
15     auto it = m_tabu_list.find(hash(i, j));
16     if (it == m_tabu_list.end()) {
17         return false;
18     }
19     return curr_iter <= it->second;
20 }

```

Listing 12: Full implementation for the key computation in the `TabuList`.

To maintain efficiency and control memory usage, the tabu list periodically executes a cleanup procedure that removes expired entries once the map grows beyond a threshold proportional to the tabu tenure and oscillation range. This keeps the data structure compact, ensuring fast lookups while avoiding excessive memory consumption.

4.2.2 Defining the Aspiration Criteria

In the context of tabu search, we occasionally allow moves that are otherwise tabu to be performed if they satisfy a condition known as an aspiration criterion. In our implementation, a tabu move can be accepted if it results in a solution whose cost is better than the best solution found so far. This mechanism ensures that the algorithm does not miss promising improvements simply because a move is temporarily forbidden, allowing the search to escape local optima and continue progressing toward higher-quality solutions.

4.2.3 Dynamic Tabu Search Behavior

Our tabu search implementation introduces a dynamic policy that adjusts the strictness of tabu restrictions depending on the search state. Specifically, a two-opt move is normally considered tabu based on the edges involved, but the exact rule depends on a flag `use_strong_policy`. When the strong policy is active, a move is forbidden even if just one of the two edges is currently tabu, enforcing a stricter exploration. Conversely, in the default, more forgiving mode, a move is only considered tabu if both edges are tabu.

The `use_strong_policy` flag itself is not static and it can be triggered dynamically when the search detects stagnation. For instance, if no improving solution has been found for a number of iterations, a “kick-start” mechanism sets `use_strong_policy = true` for a limited

duration. This temporarily forces the search to accept worse solutions and explore less-visited parts of the solution space, increasing diversity and (possibly) helping escape local optima.

4.2.4 Improving the Performance: Multi Start & Parallelization

To strengthen the tabu search and increase the chances of finding high-quality solutions, we implemented multi-start strategies that exploit multiple initial solutions. Multi-start allows us to bypass the dependency on a single starting point, which is especially useful since the quality of the initial nearest-neighbor solution can vary significantly. In our implementation, we developed two metaheuristics that both incorporate multi-start but handle it differently, trading off parallelism, filtering, and complexity.

The first approach is parallel funneling multi-start. Here, we generate a large number of candidate initial solutions using nearest-neighbor starting from different nodes, optionally refining them with a preliminary two-opt. We then select the best subset of these candidates, called survivors, and run independent tabu searches for each in parallel. This design maximizes exploration by leveraging modern multi-core hardware while funneling computational effort toward the most promising candidates. The parallel execution ensures that multiple searches proceed simultaneously without waiting on each other, improving wall-clock time efficiency.

The second approach is sequential pruning multi-start, which performs the multi-start process one candidate at a time. After generating an initial solution with nearest-neighbor, the algorithm evaluates its quality relative to the best solution found so far. If the initial tour is worse than a configurable threshold compared to the current best, the candidate is pruned and its tabu search is skipped. This filtering step compensates for the lack of parallelism by avoiding expensive tabu searches on obviously poor initial solutions. While simpler to implement, the sequential pruning approach is sensitive to the threshold choice: too lenient and computational resources are wasted on low-quality candidates; too strict and potentially strong solutions might be ignored early, especially due to “luck” in the order of starting nodes.

Both approaches exploit multi-start diversity, which increases the likelihood of escaping local optima, and they naturally integrate with the tabu search framework. Parallel funneling emphasizes exploration across promising candidates in a parallelized environment, whereas sequential pruning is simpler and focuses computational effort selectively, with the trade-off of being sensitive to threshold tuning and the sequential ordering of starts.

The code for the two algorithms is not too complex, but it’s quite long, so it is omitted from the report and remains available in the codebase for inspection.

Up to this point, our focus has been on combining two-opt local search with tabu search and multi-start strategies, applying them in a “flat” manner to the entire instance. These approaches are largely general-purpose: they do not leverage any structural information specific to the problem, beyond exploiting symmetry in the evaluation of moves. This works well for small and medium instances and even produces reasonable results on larger

instances, but the computational cost grows quickly as the instance size increases. To push efficiency further, it becomes natural to consider the intrinsic structure of our instances (components such as grids, lines, or clusters of holes) and exploit them hierarchically. The next section introduces a hierarchical tabu search, which applies the metaheuristic in a way that respects and leverages this clustered structure, improving scalability and execution times for large, structured instances, at the cost of possibly slightly worse solutions. We will analyze the trade-offs and see whether it is worth it or not.

4.3 Exploiting the Problem Structure

As observed in previous sections, particularly during the discussion on structured instance generation and the visualization of sample boards, it is evident that many boards naturally exhibit a clustered structure. These clusters, corresponding to components such as grids or lines of holes, are too significant to ignore. The idea behind exploiting the problem structure is to adopt a hierarchical approach, similar to divide-and-conquer, in which the optimization is performed at two levels: intra-cluster, to find efficient tours within each cluster, and inter-cluster, to determine the order in which clusters are visited. The tours within and between clusters are then carefully stitched together to form a complete, feasible global tour. This approach aims not only to preserve or improve solution quality but also to achieve substantial speedups compared to the current flat metaheuristic that treats the instance as unstructured.

4.3.1 Clustering Algorithm Selection

When deciding how to identify clusters in our PCB instances, we considered several common clustering algorithms: K-means, DBSCAN, and hierarchical clustering. After testing these approaches, we opted for K-means due to its simplicity, robustness across different board configurations, and efficiency.

In contrast, DBSCAN, which relies on density-based separation, proved less suitable because many points in our structured instances would be considered noise, as the hole distribution does not always form clearly dense regions. Hierarchical clustering was also considered, but its higher computational cost and complexity made it less attractive for larger instances, especially when repeated clustering would be required in the hierarchical tabu search framework. Overall, K-means offered a practical balance of speed, consistency, and reliable cluster identification for our problem.

To automatically select the number of clusters κ for each instance, we rely on the elbow method based on the silhouette score, restricting the search to the range [2,32]. This allows the algorithm to adaptively estimate a reasonable number of clusters without requiring manual inspection for every board. The clustering itself is implemented in `Python` using the `scikit-learn` library, which provides a convenient and efficient interface for `K-means`. It is important to note that, although clustering is an integral part of the hierarchical tabu search, we ignore its computation time in our performance analysis. This decision is justified

for two reasons: first, it simplifies the integration with the `C++` metaheuristic, avoiding cross-language timing issues; second, K-means is extremely fast, and even for moderately sized instances, its execution contributes negligibly to the overall runtime of the hierarchical tabu search.

A brief disclaimer is in order: we did not implement K-means ourselves, nor the silhouette-based estimation for the number of clusters. Instead, we relied on external libraries and existing functionality provided by Python’s `scikit-learn`, leveraging their tested and efficient implementations to handle clustering and K selection automatically.

A comparison between the aforementioned clustering algorithms is visually presented in Figure 5. Beyond being very fast, `k-means` also provided the most balanced number of clusters, while DBSCAN almost always produced too few, and Hierarchical Clustering too many.

For the sake of completeness, and to allow using our hierarchical tabu search also in general cases, we tried applying these clustering algorithms to random instances that by nature are not clustered. The results, however, were surprisingly good, both in terms of how the clusters look visually, as seen in Figure 6, and also in terms of performance of the actual implementation of our algorithm.

The goal of our hierarchical approach is to exploit the natural clustered structure of the board. By splitting the optimization into two phases (first focusing on intra-cluster paths and then on inter-cluster connections) we aim to capture local geometric patterns more efficiently. It is important to stress that this hierarchical approach is more restrictive than the flat tabu search. By forcing the algorithm to solve each cluster independently and then connecting them, we are limiting the solution space, which means the global optimum could be excluded, especially if the clustering happens to be “unlucky” for a given instance. The goal is therefore not necessarily to outperform the flat tabu search in solution quality, but rather to achieve solutions that are reasonably close to optimal while drastically reducing computation time, leveraging the structure of the problem to make the search more efficient.

Clustering Algorithms Comparison

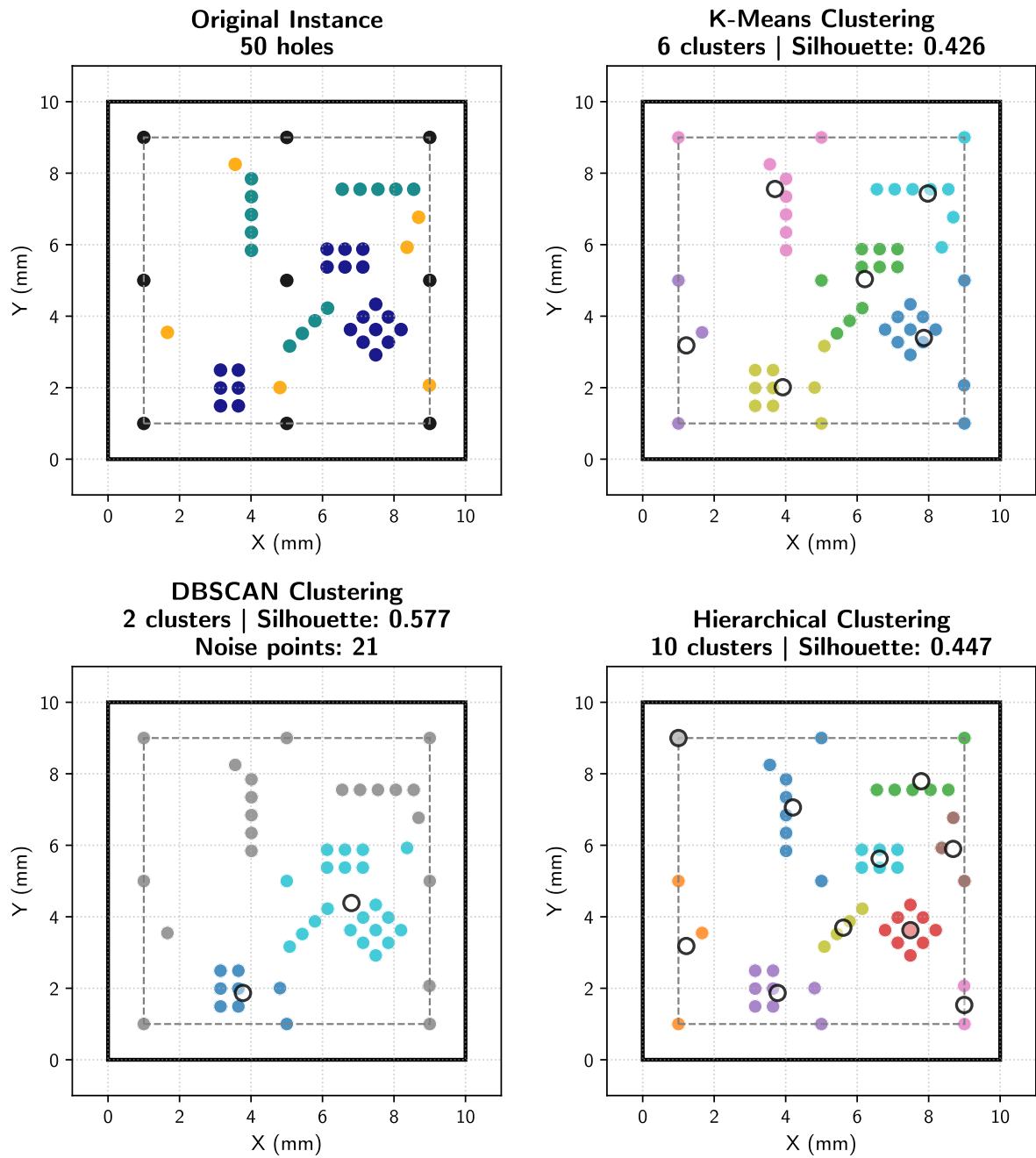


Figure 5: Comparison of different clustering algorithms for a structured instance.

Clustering Algorithms Comparison (Random Holes)

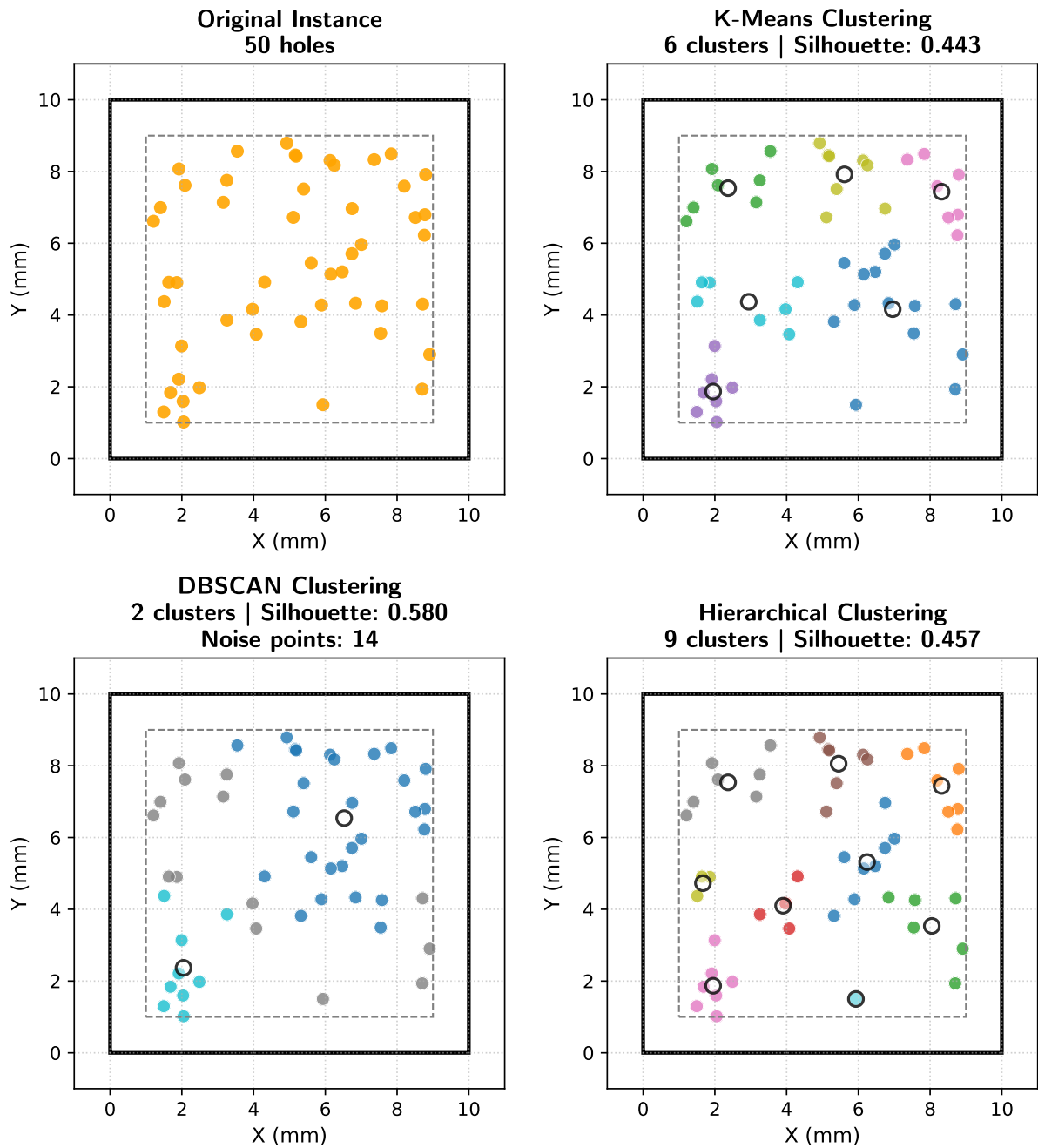


Figure 6: Comparison of different clustering algorithms for a random instance.

4.4 A Second Attempt: Hierarchical Tabu Search

A major advantage of the hierarchical approach is that we can reuse much of the code from the flat tabu search. Essentially, the problem is decomposed into smaller sub-instances corresponding to the holes in each cluster. Each sub-instance can then be solved independently using the same flat tabu search already developed, with the key benefit that smaller instances are much faster to optimize. The same principle applies when determining the order in which clusters themselves are visited, although this requires a careful definition of inter-cluster distances to treat clusters geometrically. The truly novel part of the hierarchical tabu search is the logic that allows us to stitch together the intra-cluster and inter-cluster solutions into a single feasible tour. This step is both essential and non-trivial, and it is what enables the hierarchical method to exploit the problem structure effectively.

4.4.1 Inter-Cluster Optimization

For the inter-cluster optimization, a crucial step is deciding how to represent clusters geometrically. Our first attempt relied on centroids, treating each cluster as a single point at the average location of its holes. While this is conceptually simple, it performed poorly in practice. The centroid-based ordering often produces highly suboptimal cluster sequences, because clusters are rarely compact: two clusters whose centroids are far apart may have boundary points that are very close, allowing a cheap connection that the centroid-based approach completely misses.

To address this, we switched to a minimum-boundary distance approach. Here, the distance between two clusters is defined as the shortest distance between any pair of points, one from each cluster. This method better captures the true geometric relationship between clusters and leads to more reasonable inter-cluster orderings. The downside is higher computational complexity, since all pairwise distances between points of different clusters must be evaluated. However, this cost is incurred only once at the beginning, effectively as part of instance preprocessing, and does not affect the runtime of the hierarchical tabu search itself. This trade-off was deemed acceptable, as it dramatically improves the quality of inter-cluster sequencing without slowing down the actual optimization.

For the actual hierarchical optimization, we leverage the sequential pruning multi-start approach introduced earlier, but applied at the cluster level. Specifically, we generate multiple inter-cluster tours, aiming to obtain the top- k cluster orderings, where k is a tunable parameter.

Each ordering is treated as a separate starting point for stitching the intra-cluster tours computed with the flat tabu search. This allows us to explore different ways of connecting clusters, compensating for the fact that even the minimum-boundary distances do not guarantee a globally optimal inter-cluster sequence. By trying multiple high-quality cluster orderings, the hierarchical approach combines local intra-cluster optimization with multiple inter-cluster connection strategies, increasing the likelihood of finding a near-optimal global tour while keeping the computational cost manageable.

4.4.2 Intra-Cluster Optimization

For intra-cluster optimization, we treat each cluster as a standalone instance and apply the sequential pruning multi-start tabu search within it. The key advantage here is that, since clusters are independent at this stage, we can parallelize the optimization across clusters, computing the best intra-cluster tours simultaneously. Each cluster’s tour is optimized locally, benefiting from the small size of the cluster (faster tabu search), while maintaining the flexibility to later combine these tours according to the multiple inter-cluster orderings generated in the previous step. This parallelized intra-cluster optimization significantly reduces runtime without compromising the quality of the cluster-level solutions.

4.4.3 Stitching the Cluster Tours to Obtain a Global Tour

Clusters are solved as standalone subproblems, meaning that the heuristic produces a closed tour for each cluster. When assembling a global tour, however, each cluster tour must be converted into an open path that can be connected to the preceding and succeeding clusters in a feasible and cost-effective way. This transition from local closed tours to a global open structure is non-trivial, both conceptually and algorithmically.

A first natural idea is a greedy constructive approach. Given an ordering of clusters, one could select, for each cluster, an entry point that minimizes the distance from the exit point of the previous cluster. This can be implemented by simply rotating the cluster tour so that it starts at the node closest to the previous cluster’s exit. While appealing in its simplicity, this strategy performs poorly in practice.

The underlying issue stems from the fact that each cluster tour is optimized in isolation. A locally optimal tour is typically tight: the first and last nodes are close to each other. This is precisely the desired outcome when solving the problem locally, but it becomes problematic when the tour must be embedded into a larger global structure.

If the tour is simply rotated to select an entry point that minimizes the connection cost to the previous cluster, the exit point is implicitly fixed as well. Since the tour is a rigid cycle, the last node of the rotated sequence will also be close to the entry point, and therefore close to the previous cluster rather than to the next one. As a consequence, the connection to the following cluster is often long and costly. Trying to optimize for the following cluster as well as the previous, by introducing lookahead, does not fully resolve this issue, as the entry point of the next cluster is itself unknown at that stage, leading to a chicken-and-egg dependency.

A more effective approach exploits a simple but crucial observation: a closed tour within a cluster becomes an open path in the global tour. Rather than rotating a rigid cycle, we can break exactly one edge of the cluster tour and use its two endpoints as the entry and exit points of the cluster. This operation introduces two free endpoints, which can be assigned asymmetric roles: one optimized for the connection to the previous cluster, and the other optimized for the connection to the next. In symmetric instances a given tour can be traveled in two directions (with no difference in cost), allowing each of the two end nodes of a single edge to assume the role of either the entry or the exit node. This gives us

an additional degree of freedom: since we can decide which end node (of some fixed edge) to use as the entry node and which one to use as the exit node, we can just select as entry node the one that is closest to the previous cluster, or select as exit node the one that is the closest to the next cluster. Note that we cannot hope to select two nodes satisfying both conditions, as we are constrained to choosing two nodes that are actually connected by an edge in the tour within the current cluster.

The central question then becomes which edge to break. Choosing an edge at random is clearly suboptimal. Instead, a clever idea is to break the edge whose removal yields the best overall connection to the surrounding clusters. Concretely, for a cluster positioned between two others in the global ordering, we evaluate each edge of its tour as a candidate break point.

Although this requires examining all edges of the cluster tour, the computational cost remains low in practice. Clusters are significantly smaller than the full instance, and the cost variation induced by breaking an edge can be computed in constant time $\mathcal{O}(1)$, by subtracting the cost of the removed edge and adding the costs of the two new inter-cluster connections. This makes the approach both effective and computationally efficient, while properly accounting for the interaction between neighboring clusters.

A final clarification is required regarding what it concretely means to connect the current cluster to its predecessor and successor when evaluating which internal edge to break. For the previous cluster, the situation is simple: the connection is determined by the last node of the current global tour, which coincides with the exit node of the cluster optimized in the preceding iteration. This node is already fixed and can be used directly when evaluating candidate break points in the current cluster.

The connection to the following cluster, however, is less straightforward, as that cluster has not yet been optimized. This introduces an unavoidable lookahead component. For each candidate edge to be broken in the current cluster, and for each of the two possible assignments of its endpoints as entry and exit, we approximate the future connection by considering the node in the following cluster that is closest to the candidate exit node.

Although approximate, this strategy yields a solution that is sufficiently informative for decision-making. Moreover, it has a beneficial side effect: by selecting an exit node that is already close to some node in the next cluster, we implicitly provide that cluster with a favorable incoming connection. When the following cluster is subsequently optimized, this advantage naturally propagates, often resulting in a short and well-aligned inter-cluster bridge. In this way, the limited lookahead not only guides the current decision but also facilitates the optimization of the next cluster, without incurring the full cost of a global inter-cluster search.

The underlying logic of the algorithm is notably elegant: a single, well-chosen edge removal transforms an overconstrained local tour into a flexible component that can adapt optimally to its global context. However, the code to implement this is quite long. For this reason, rather than presenting lengthy and distracting source code, we provide only a high-

level pseudocode description that captures the essential mechanics of the method while keeping the focus on the algorithmic insight rather than its implementation overhead.

```

1 Initialize  $T_{\text{global}}$  with the tour of the first cluster  $C_1$ 
2  $u_{\text{curr}} \leftarrow$  last node of  $T_{\text{global}}$ 
3 for each cluster  $C_k$  in  $C_{\text{ordering}}$  (from  $k = 2$  to  $N_{\text{clusters}}$ ) do
4   Let  $T_C$  be the closed tour of cluster  $C_k$ 
5   Let  $S_{\text{next}}$  be the set of nodes in cluster  $C_{k+1}$  (or  $\{T_{\text{global}}[0]\}$  if  $k = N_{\text{clusters}}$ )
6    $\delta^* \leftarrow +\infty$ ,  $e^* \leftarrow \text{none}$ ,  $d^* \leftarrow \text{none}$ 
7   for each edge  $(u, v)$  in  $T_C$  do
8      $c_{\text{broken}} \leftarrow \text{cost}(u, v)$  /* shared for both directions */
9
10    /* forward direction: enter at  $u$ , exit at  $v$  */
11     $w_{\text{target}} \leftarrow \arg \min_{w \in S_{\text{next}}} \text{cost}(v, w)$ 
12     $\delta_{\text{fwd}} \leftarrow \text{cost}(u_{\text{curr}}, u) + \text{cost}(v, w_{\text{target}}) - c_{\text{broken}}$ 
13
14    /* reverse direction: enter at  $v$ , exit at  $u$  */
15     $w_{\text{target}} \leftarrow \arg \min_{w \in S_{\text{next}}} \text{cost}(u, w)$ 
16     $\delta_{\text{rev}} \leftarrow \text{cost}(u_{\text{curr}}, v) + \text{cost}(u, w_{\text{target}}) - c_{\text{broken}}$ 
17
18    if  $\min(\delta_{\text{fwd}}, \delta_{\text{rev}}) < \delta^*$  then
19      | Update  $\delta^*$ ,  $e^*$ , and direction  $d^*$ 
20    end if
21  end for
22  Break edge  $e^*$  in  $T_C$  and orient path according to  $d^*$ 
23  Append path to  $T_{\text{global}}$ 
24   $u_{\text{curr}} \leftarrow$  last node of  $T_{\text{global}}$ 
25 end for
26 return  $T_{\text{global}}$ 

```

Algorithm 1: Procedure to stitch cluster tours into a global tour.

4.4.4 Polishing the Stitched Global Tour

After the stitching phase, we apply a final polishing step to the global tour by running a standard best improvement two-opt local search on the entire instance. The goal is to allow for small adjustments and refinements that were not visible when optimizing each cluster in isolation, but that become apparent once the full tour structure is fixed. Importantly, this operation does not disrupt the overall organization of the solution: due to the way the global tour is constructed, the solution lies deep inside a local optimum basin, and large-scale rearrangements are extremely unlikely. As a result, the two-opt phase mainly smooths minor inconsistencies at cluster boundaries and occasionally recovers small improvements.

4.4.5 Parameter Configuration & Tuning

After implementing the algorithm, and before being able to exploit it effectively, it is necessary to select appropriate values for the parameters that characterize its behavior. In our case, the most critical parameter of the tabu search is the tabu tenure, which determines for how many iterations a move remains forbidden before becoming available again. The optimal value of the tabu tenure strongly depends on the type and size of the instances under consideration. Small instances often benefit from low or moderate tenure values, while larger instances, which admit a vastly larger neighborhood of possible moves, typically require higher tenures to avoid short cycling and premature looping.

Since no single value can be fixed once and for all, the tabu tenure must be actively tuned by testing different configurations on a common set of training instances and evaluating their overall performance. This tuning phase is one of the most important steps in the design of a tabu-search-based heuristic, as it often makes the difference between obtaining a good solution and settling for a merely acceptable one. Ideally, a reasonable amount of time should be devoted to this phase, for instance through a grid search over plausible parameter ranges.

Our implementation also includes a dynamic tabu tenure mechanism, where the tenure oscillates around a default value within a prescribed range. This introduces an additional parameter that would, in principle, require tuning. For simplicity, we fixed the oscillation amplitude to half of the base tabu tenure. This choice is admittedly arbitrary and not supported by a strong theoretical justification. While it is likely suboptimal for certain classes of instances, it represents a pragmatic compromise aimed at limiting the overall tuning effort.

Beyond the tabu tenure, several additional parameters arise from the overall heuristic framework rather than from the tabu search itself. A notable example is the number of multistart initializations. Choosing too few starting solutions increases the risk of getting trapped in poor regions of the solution space, while choosing too many can lead to redundancy and unnecessary computational effort. As with the tabu tenure, the appropriate number of starts depends on the instance size and structure. Small instances typically require only a limited number of starts, whereas larger instances benefit from a broader exploration of the solution space. In this context, it is also important that the starting solutions be sufficiently diverse: selecting starting points that are too similar undermines the purpose of multistart.

Another parameter is the threshold used for sequential pruning, which determines when an initial solution generated by the nearest-neighbor heuristic is discarded because it is excessively worse than the best solution found so far. This parameter is particularly delicate, as it does not depend solely on instance size: even instances of identical size may require different thresholds depending on their geometric structure. For this reason, a conservative choice is preferable, allowing slightly worse initial solutions to pass the filter in order to avoid prematurely discarding promising regions of the search space.

Finally, the maximum number of tabu search iterations and the associated patience parameter, defined as the number of consecutive non-improving iterations tolerated before termination, play a crucial role. Increasing these values generally improves solution quality, but only up to a point. Beyond that, improvements become marginal and are no longer justified by the additional computational cost, leading to diminishing returns.

As this discussion makes clear, parameter tuning is inherently challenging, especially when multiple interacting parameters are involved and each admits a wide range of plausible values. While a comprehensive grid search would be the most reliable approach, it is also computationally expensive. For this reason, we deliberately limit the time spent on parameter tuning, accepting a potential loss in solution quality as a necessary trade-off. In practice, we select parameter values as simple functions of the instance size, without attempting an exhaustive optimization of the parameter space.

4.5 Running the Heuristic Solver

Once the heuristic framework has been fully implemented and its parameters selected, we can turn to the execution of the heuristic solver. At this stage, the **C++** code operates as a self-contained program that reads a preprocessed instance (the cost-matrix **.dat** file and the cluster distribution **.clusters** file) and applies tabu search (with the different variants we have explored) to compute a high-quality routing solution.

As for the exact solver, compilation and execution follow a **Makefile**-based workflow.

4.5.1 Compilation

Before running the heuristic solver, the code must be compiled. The implementation requires a **C++** compiler supporting at least the **C++11** standard, such as **g++**. Unlike the SCF solver, no external optimization libraries are required, as all heuristic components are implemented directly in **C++**.

The compilation is handled analogously to that of the SCF Solver, using **make**. The appropriate commands can be found in 3.4.1.

If the compilation completes successfully, the **bin/** directory will contain the executables **single_instance_heuristic_solver** and **batch_heuristic_solver**.

4.5.2 Solving an Instance

To solve a single instance, the **single_instance_heuristic_solver** executable is used. The general command-line syntax is as follows:

```
$ ./bin/single_instance_heuristic_solver <filename.dat> <filename.clusters> \  
    [--verbose <0|1|2>] [--average <n_runs>] [--print-tours]
```

The program arguments have the same meaning as in the SCF Solver. The only differences are that our heuristic solver also requires the **.clusters** file in order to be able to run the hierarchical tabu search and that an additional optional parameter **--print-tours** can be provided, in order to also print the tours corresponding to the different algorithms.

4.5.3 Batch Solver

The batch solver provides an automated way to solve multiple instances in a single execution. Instead of operating on a single instance (pair of `.dat` and `.clusters` files), the program takes as input a pair of directories containing the corresponding files for the cost-matrix and the clusters of many instances. The command syntax for the executable is the following:

```
$ ./bin/batch_heuristic_solver <costs_dir> <clusters_dir> \  
    [--average <n_runs>] [--output <filename>]
```

The two mandatory arguments specify the input directories. The first, `costs_dir`, must contain the `.dat` files describing the cost matrices of the instances. The second, `clusters_dir`, must contain the files describing the cluster decompositions associated with those same instances. The solver matches instances across the two directories by filename, ignoring the file extension. In other words, each cost matrix file is paired with the cluster file that shares the same base name.

For example, suppose the directory `costs_dir` contains the files `instance_01.dat` and `instance_02.dat`, while the directory `clusters_dir` contains the files `instance_01.clusters` and `instance_02.clusters`. In this case, the solver will correctly associate `instance_01.dat` with `instance_01.clusters`, and `instance_02.dat` with `instance_02.clusters`, and then process each matched pair as an independent instance.

The optional `--output` parameter specifies the name of the file where the aggregated results are written. If no output filename is provided, the solver automatically creates a file named `results.dat` in the working directory and writes all results to it.

This chapter presents the experimental evaluation of the methods introduced in this work. We begin by describing the benchmark instances and the experimental setup used throughout the study, clarifying how data were generated and grouped. We then analyze the performance of the exact SCF solver implemented in CPLEX, focusing on the computational cost in terms of execution times. This is followed by an evaluation of the proposed heuristic algorithms, highlighting their scalability and practical effectiveness on larger instances. The two approaches are then compared directly, in order to assess the trade-offs between optimality and efficiency. Finally, we move beyond aggregate numerical results and examine the structure of the resulting tours, providing qualitative insights into how different methods shape the final solutions.

5.1 Benchmark Instances and Experiments

Before presenting the experimental results, a brief disclaimer is in order. Over the course of this work, the modeling and algorithmic choices accumulated to the point where the space of possible instance configurations and parameter combinations became extremely large. While this flexibility is powerful, it also makes it impractical to exhaustively explore all possible benchmarks within reasonable time and computational limits. As a result, compromises were unavoidable: there are infinitely many valid ways to construct benchmark datasets, and the ones adopted here represent a careful but necessarily limited selection.

We restrict our attention to two broad classes of instances: random instances and structured instances. Random instances consist of holes placed uniformly at random on the board. They serve as a useful worst-case baseline, allowing us to assess the raw performance of both the exact and heuristic methods in the absence of exploitable structure. In this sense, they reflect a generic routing problem rather than the specific industrial scenario motivating this work. At the same time, they provide a reference against which the benefits of structure-aware approaches can be evaluated.

Structured instances, on the other hand, are designed to reflect the type of organization encountered in realistic drilling boards, where holes are arranged according to repeated or clustered patterns. While these instances are still simplified and optimized for experimental clarity rather than full realism, they capture the operational structure that the proposed hierarchical and clustering-based heuristics are intended to exploit. An additional advantage is that such instances are easier to visualize and interpret, which proved valuable when analyzing and comparing tour structures.

For both instance types, multiple instances were generated and grouped by size, measured in terms of the number of nodes. In the case of structured instances, a fixed high-level configuration is used for all instances of the same size. Variability is introduced

through randomized geometric placement of the components, ensuring that each instance has a distinct spatial layout and topological structure. This allows us to obtain more robust results while preserving structural consistency within each size class.

More specifically, random instances were generated with sizes ranging from 10 to 100 nodes. All instances are embedded in a board of size 100×100 units with a drilling head speed fixed at 100 units per second (along both axes, for simplicity).

For structured instances, achieving the same level of granularity is inherently more difficult. Even simple structured layouts require a minimum number of nodes, making very small instances impractical or meaningless. For this reason, structured instances were generated only at sizes 30, 50, 75, and 100. While less granular than the random case, these checkpoints are sufficiently well distributed to capture the main performance trends.

Two final remarks are important for interpreting the results. First, the choice of instance sizes is constrained by computational resources and time limitations. The selected benchmarks are suitable for comparative analysis but are not extensive enough to support a detailed statistical study of fine-grained performance patterns. With more time and resources, a significantly larger and more diverse dataset, including variations in board size and density, would be desirable.

Second, these limitations affect the exact and heuristic methods differently. Solving instances larger than 100 nodes with the SCF model in CPLEX was deemed prohibitively expensive, especially when averaging over multiple runs for robustness. However, restricting the heuristic algorithms to the same size range would fail to demonstrate their intended scalability. For this reason, additional large structured instances, with sizes up to 1000 nodes, were considered exclusively for the heuristic setting. For these instances, optimal solutions are out of reach for CPLEX, and performance is instead evaluated relative to the Held–Karp lower bound, computed using external tools. Consequently, the results for large instances should not be interpreted in terms of absolute optimality gaps, but rather as indicators of qualitative trends and scaling behavior as the instance size grows.

5.2 CPLEX SCF Solver Performance

We now turn to the performance of the SCF model solved with CPLEX, focusing on execution times and on how the objective value evolves under different cost metrics and instance characteristics.

5.2.1 Execution Times

We run the CPLEX SCF Solver on many different instances in terms of type and size. The results of the execution times on random instances are shown in Figure 7 and detailed in Table 3. For smaller instances, CPLEX is highly effective. Up to 60 holes, the average solve time remains under 5 seconds. The variance in this region is relatively low, indicating stable performance regardless of the specific random geometric configuration.

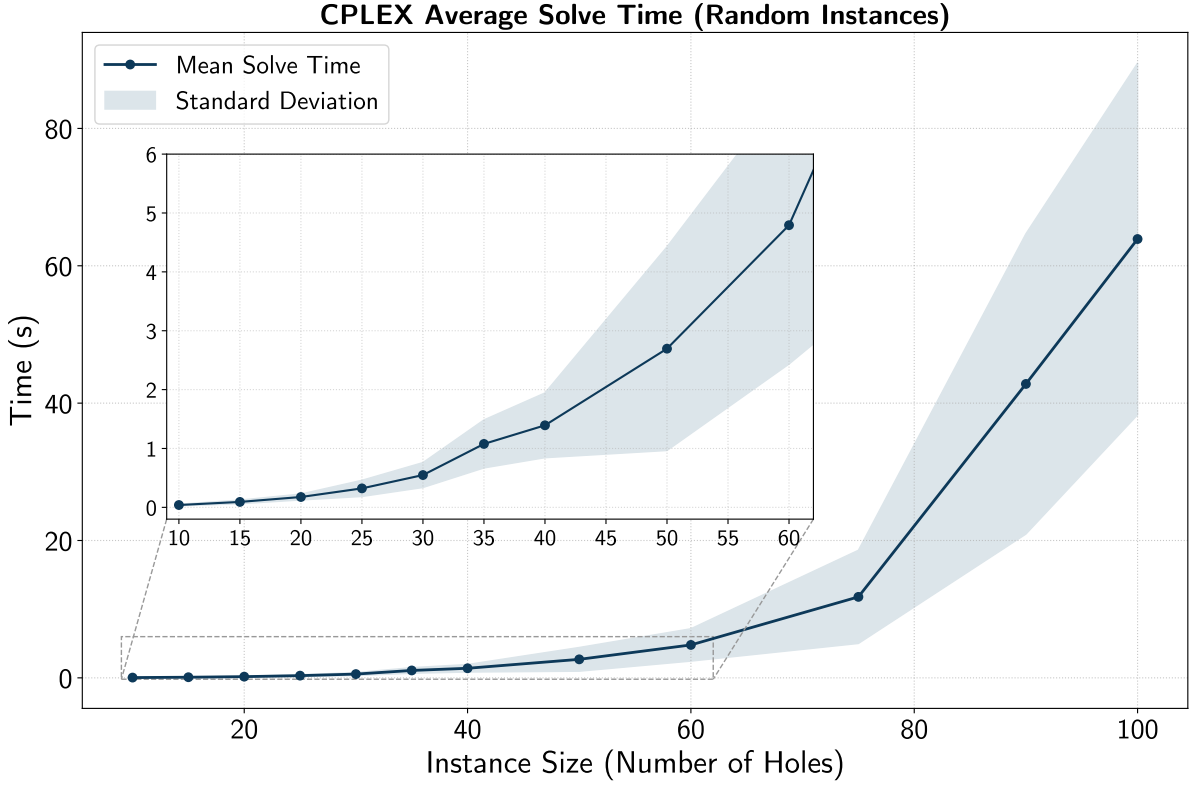


Figure 7: CPLEX Performance (Average Solve Time) over random instances as a function of instance size.

Size (# Holes)	Avg. Model Creation Time (s)	Avg. Solve Time (s)	Standard Deviation (s)	Min. Solve Time (s)	Max. Solve Time (s)
10	0.0002	0.0397	0.0112	0.0144	0.0594
15	0.0004	0.0924	0.0205	0.0391	0.1219
20	0.0007	0.1759	0.0341	0.1234	0.2498
25	0.0009	0.3222	0.0971	0.2140	0.5719
30	0.0013	0.5487	0.1565	0.3488	0.8670
35	0.0017	1.0773	0.3159	0.5692	1.8168
40	0.0024	1.3934	0.4586	0.8451	2.1433
50	0.0035	2.6952	1.3768	1.4770	5.7130
60	0.0055	4.7929	1.8068	1.9957	8.5478
75	0.0095	11.7911	4.9881	4.3311	22.4429
90	0.0125	42.7949	19.3450	18.5726	63.5550
100	0.0165	63.8964	17.0981	39.4295	89.6581

Table 3: CPLEX Performance on multiple instance sizes. Each instance size shows the average results over 10 different (random) instances in a board of size 10 mm $\times$ 10 mm, with the drill head speed equal to 10 mm/s for both axes.

However, a distinct transition to intractability appears around $N = 75$, where the average solve time jumps to 11.8s. Beyond this point, the computational cost escalates rapidly, reaching an average of 63.9s at $N = 100$. Crucially, the shaded region in Figure 7, representing standard deviation, widens dramatically beyond $N = 75$. At $N = 100$, the

standard deviation reaches 17.1s, with worst-case solve times approaching 90 seconds. This unpredictability suggests that for large-scale industrial PCBs an exact solver would be computationally prohibitive and inconsistent, with times that heavily depend on the geometric patterns of the holes.

We further tested CPLEX on structured instances to determine if the geometric regularity typical of real-world components aids the exact solver.

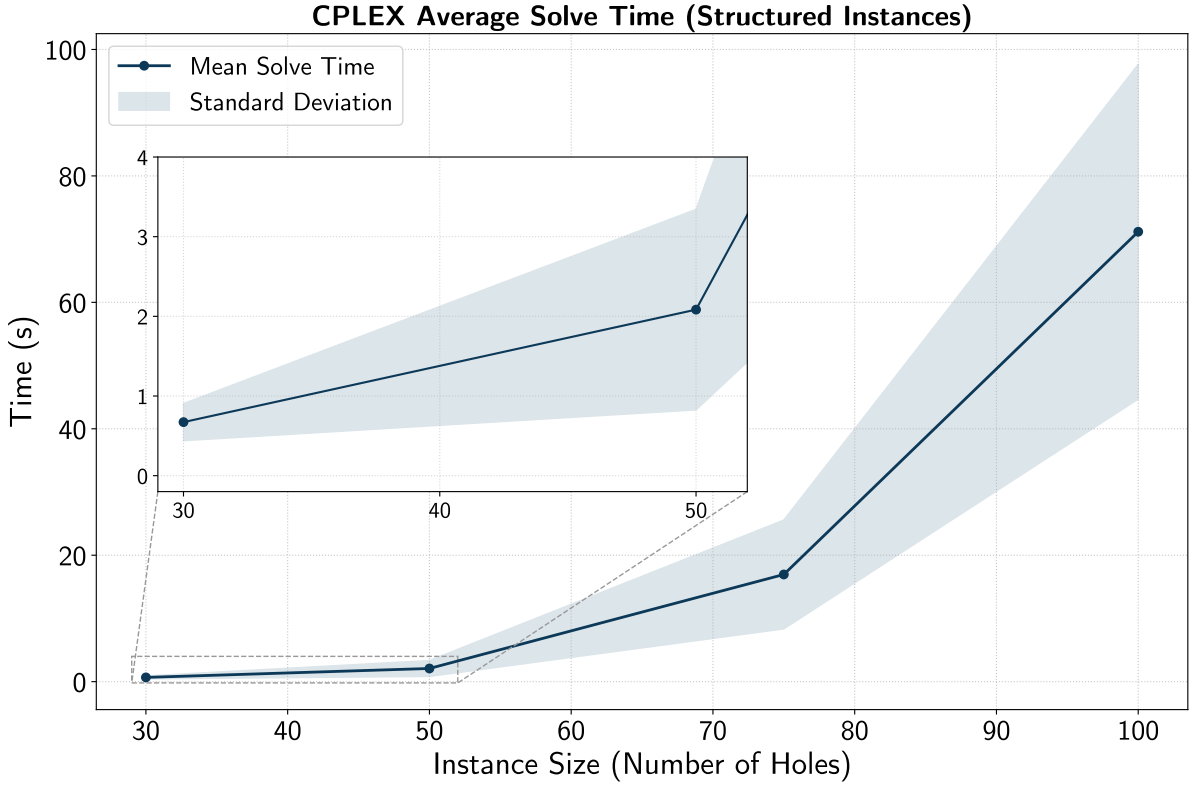


Figure 8: CPLEX Performance (Average Solve Time) over structured instances as a function of instance size.

Size (# Holes)	Avg. Model Creation Time (s)	Avg. Solve Time (s)	Standard Deviation (s)	Min. Solve Time (s)	Max. Solve Time (s)
30	0.0025	0.6722	0.2334	0.4081	1.1669
50	0.0036	2.0843	1.2588	0.6934	3.9850
75	0.0089	16.9536	8.6117	6.0767	28.3873
100	0.0164	71.1681	26.4739	15.8849	104.5300

Table 4: CPLEX Performance on multiple instance sizes. Each instance size shows the average results over 10 different (structured) instances in a board of size 10 mm $\times$ 10 mm, with the drill head speed equal to 10 mm/s for both axes. For each instance size, the instances have the same structured components, and they differ only on the geometric placement within the board.

The results, shown in Figure 8 and detailed in Table 4, reveal a counter-intuitive finding: structural constraints do not necessarily simplify the problem for standard branch-and-cut solvers. For structured instances at $N = 100$, the average solve time increased to 71.17s, compared to 63.90s for random instances. More importantly, the instability of the solver worsened significantly, with the standard deviation rising to 26.47s. The maximum solve time observed for a structured $N = 100$ instance was 104.53s. This suggests that the combination of dense point clusters (representing chips) and sparse connections (between components) may create “symmetry traps” or difficult branching decisions for CPLEX, making the search space harder to prune effectively than in uniformly random distributions.

The CPLEX baseline demonstrates that while exact solution methods are viable for small prototyping boards, they face severe scalability barriers. Still, an important remark is in order. Exact solvers like CPLEX often identify near-optimal solutions relatively quickly but consume the vast majority of their runtime closing the final optimality gap. The process of proving the optimality of a solution is not irrelevant and requires exploring and pruning vast regions of the search space, even when no further improvement to the objective value is likely.

Heuristic methods operate under a fundamentally different paradigm. Since they are not required to provide optimality guarantees, they can devote their entire computational budget to exploring promising regions of the solution space and refining good solutions. This distinction is crucial when comparing execution times: the longer runtimes observed for CPLEX do not only reflect solution difficulty, but also the intrinsic cost of optimality certification, a cost that heuristics deliberately avoid.

5.2.2 Comparing Metrics

In addition to measuring execution times, we also analyze how the choice of distance metric affects the objective value produced by the SCF solver. Different metrics correspond to different assumptions on the kinematics of the drilling head. In our work, we have considered three different metrics: L_1 (the Manhattan distance), L_2 (the Euclidean distance), and L_∞ (the Chebyshev distance). The results are shown in Figure 9.

To make the comparison meaningful across instance sizes, the objective values obtained under L_1 and L_∞ are normalized with the respect to the L_2 objective, which is used as a reference baseline.

For all instance sizes considered, the L_1 metric consistently yields higher objective values than L_2 , reflecting the fact that Manhattan distances penalize diagonal movement and therefore produce longer tours. Conversely, the L_∞ metric yields lower objective values, since the drill head is allowed to move at maximum speed along each axis. The results align well with intuition and theoretical expectations: they remain stable regardless of instance size. This stability indicates that the impact of the distance metric is largely independent of the problem size and instead reflects intrinsic geometric properties of the metrics themselves.

Given the consistency of these results, we can safely restrict the remainder of the analysis to the Euclidean L_2 metric. Using L_2 as the reference simplifies both the presentation and

the interpretation of the results, without loss of generality or insight. The conclusions drawn from L_2 -based experiments naturally extend to the other movement models, which differ mainly by a predictable scaling factor rather than by qualitative behavior.

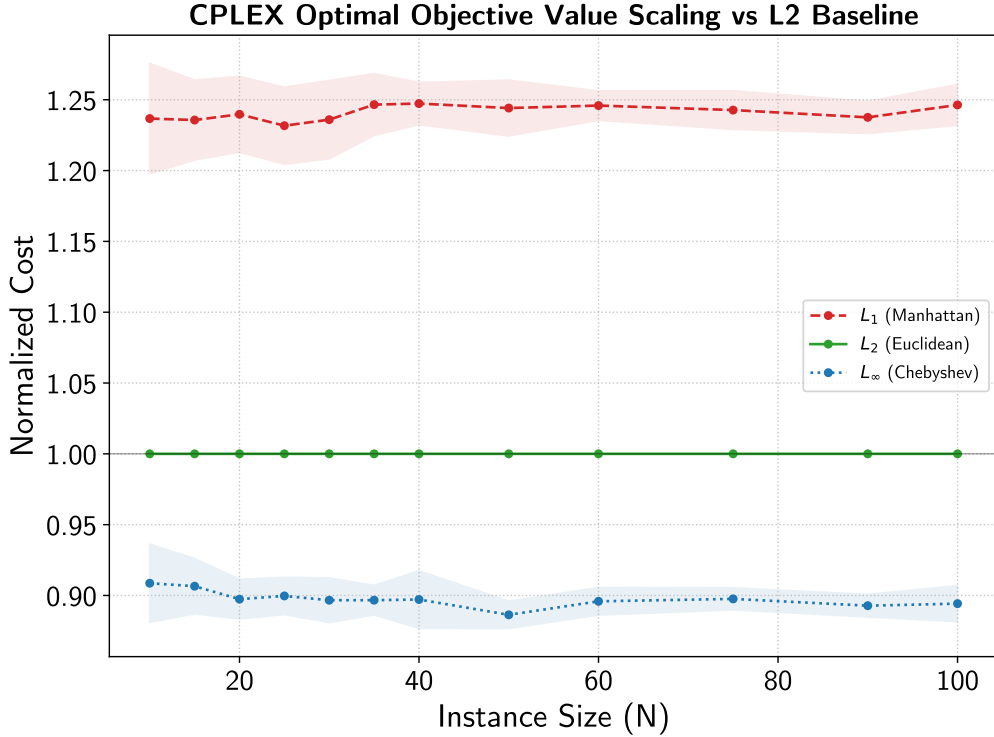


Figure 9: Objective value comparison across different distance metrics induced by different assumptions on the movement of the drill head.

5.3 Heuristic Algorithms Performance

We now turn to the analysis of the heuristic algorithms developed throughout the previous chapter. To keep the presentation readable and avoid repeating long algorithm names, we introduce the following shorthand notation:

- F.S. → Flat Sequential Pruning
- F.P. → Flat Parallel Funneling
- H.S. → Hierarchical Sequential
- H.P. → Hierarchical Parallel

All four methods are heuristic solvers, differing in how they generate initial solutions, exploit parallelism, and leverage the structure of the PCB board.

The flat approaches operate directly on the full instance. In F.S., multiple initial solutions are generated using a nearest-neighbor heuristic and refined sequentially via tabu search; poor initial solutions are discarded early if their cost exceeds a fixed threshold relative to the best solution found so far. F.P. extends this idea by first generating a large pool of candidate initial solutions, selecting the top- k , and optimizing them in parallel, replacing early pruning with a funnel-style selection phase.

The hierarchical approaches explicitly exploit the board’s component structure. Both H.S. and H.P. split the problem into layers: first, an ordering of clusters (components) is optimized; then, tours are optimized within each cluster; finally, the cluster tours are stitched together into a global tour using a smart stitching procedure.

As in the CPLEX SCF Solver evaluation, all heuristic variants are tested across a range of benchmark instances, spanning both random and structured layouts, with sizes ranging from 10 to 100 nodes. This allows us to assess not only raw performance, but also how structural awareness and parallelization affect solution quality and scalability across different problem regimes.

In this section, we compare the performance between the heuristic algorithms and use the Held-Karp lower bound as reference for computing the gaps. The next section will provide further insights with a comparison of the heuristic algorithms and the CPLEX SCF Solver.

5.3.1 Speedup & (Pessimistic) Optimality Gaps

The evaluated heuristics are compared along two orthogonal dimensions: computational efficiency and solution quality. Computational efficiency is measured as speedup relative to the Flat Sequential pruning heuristic (F.S.), which serves as the baseline. Solution quality is assessed through the average optimality gap with respect to the Held–Karp lower bound for the Traveling Salesman Problem.

It is important to note that the Held–Karp bound constitutes a lower bound on the optimal tour length. Consequently, the reported gaps represent upper bounds on the true optimality gap. While this may lead to slightly pessimistic absolute values, it ensures a consistent and fair comparison across all heuristics.

For random instances, the results are shown in Figure 10 and detailed in Table 5.

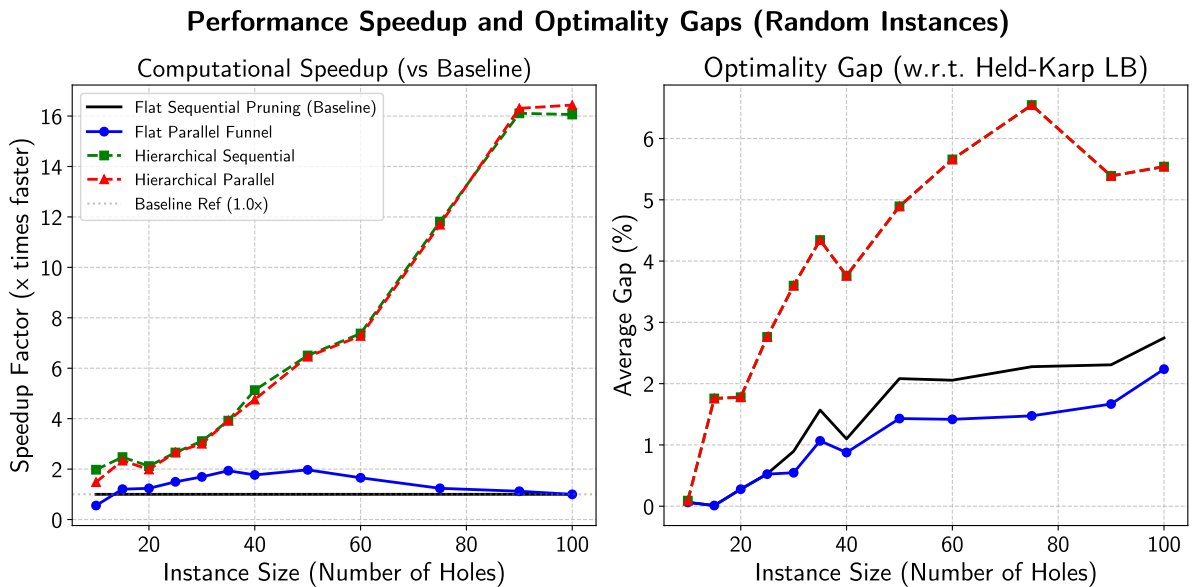


Figure 10: Heuristic performance summary for random instances. Speedups relative to Flat Sequential Pruning. Gaps relative to Held-Karp lower bound.

The Flat Sequential (F.S.) heuristic exhibits steadily increasing runtime as the number of holes grows, reaching approximately 0.02 seconds for instances with 100 holes. The Flat Parallel Funnel (F.P.) achieves modest speedups, generally ranging between $1\times$ and $2\times$.

In contrast, the Hierarchical Sequential (H.S.) heuristic demonstrates a significantly improved scaling behavior. Speedups increase rapidly with instance size, exceeding $10\times$ for instances with more than 75 holes. The Hierarchical Parallel (H.P.) variant shows a similar behavior. In this case, the benefit of using a parallel approach is not so evident, most likely because the instance size is too small to make the overhead of handling threads insignificant with respect to the total solve time, which remains very low.

Regarding solution quality, both flat heuristics maintain relatively small optimality gaps, remaining below approximately 3% across all tested sizes. The hierarchical approaches exhibit larger gaps, which increase gradually with instance size, stabilizing around 5–6%. This behavior reflects the tradeoff introduced by hierarchical decomposition, which restricts the solution space through cluster-based optimization.

When applied to structured instances that reflect realistic PCB layouts, the behavior is similar. The benefits don't get much better compared to random instances, but this comes from the fact that instances up to 100 nodes are too small in order for the hierarchical approach to be really effective. The results are shown in Figure 11 and detailed in Table 6.

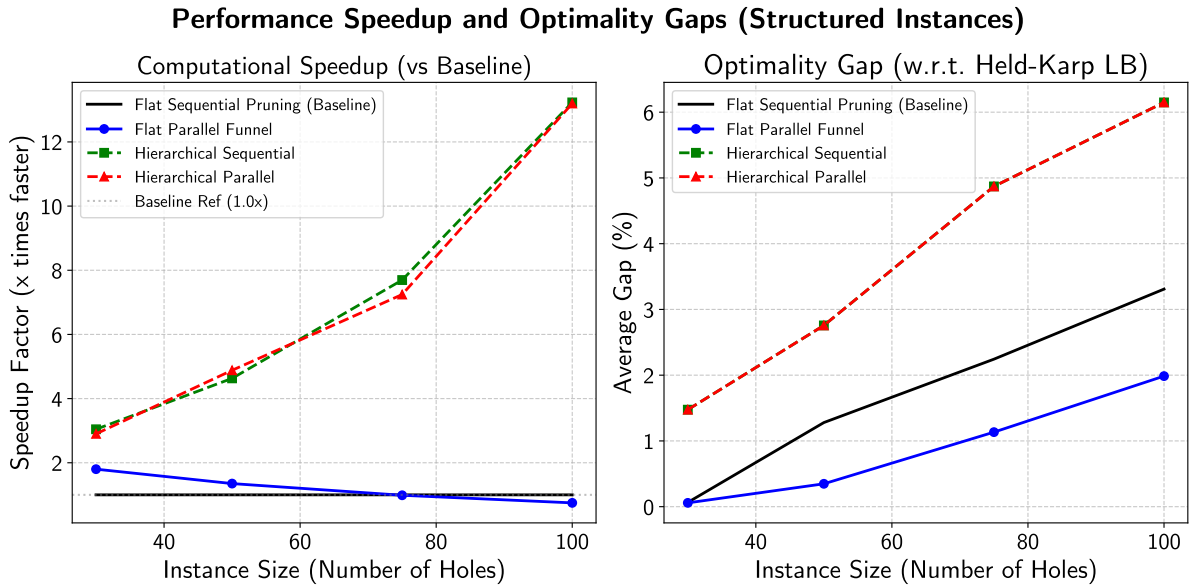


Figure 11: Heuristic performance summary for structured instances. Speedups relative to Flat Sequential Pruning. Gaps relative to Held-Karp lower bound.

Overall, for instances up to 100 nodes, the Flat Parallel funnel (F.P.) dominates Flat Sequential pruning (F.S.), achieving similar execution times (with small differences in both directions) and consistently smaller optimality gaps. This behavior stems from the ability of F.P. to explore a broader set of initial solutions in parallel and to concentrate computational effort on the most promising candidates from the outset. In this regime, the overhead of pruning in F.S. outweighs its benefits, causing it to discard potentially useful trajectories too early while still lacking a sufficiently strong incumbent to guide the pruning effectively.

As a consequence, F.S. emerges as the weakest performer among the flat heuristics for small and medium-sized instances.

Moreover, the advantages of the H.P. over the H.S. are not so evident, making the two algorithms perform almost exactly the same.

However, as mentioned before, the benchmark on small instances hides something that is clear when the instance sizes start to grow significantly. The results on bigger structured instances are shown in Figure 12 and detailed in Table 7.

For large structured instances containing up to 1000 holes, the limitations of flat heuristics become pronounced. The Flat Parallel Funnel provides only marginal speedups, rarely exceeding $3\times$, while the Flat Sequential baseline experiences rapidly increasing runtimes. The hierarchical heuristics demonstrate a fundamentally different scaling behavior. The sequential hierarchical method achieves speedups between $25\times$ and $40\times$, while the parallel hierarchical heuristic reaches speedups above $60\times$ for the largest instances. Although optimality gaps increase slightly for larger instances, they remain bounded and stable, generally below 7.5%.

Another interesting trend is revealed upon benchmarking on bigger instances: while the execution time advantage of F.P. remains essentially unchanged, the behavior of the optimality gaps reverses: F.S. begins to produce better solutions than F.P. In this setting, the pruning mechanism becomes increasingly effective, as the growing search space amplifies the benefit of aggressively eliminating low-quality candidates and focusing the sequential search around a progressively stronger incumbent.

5.3.2 The Impact of Parallelization and Structural Decomposition

The experimental results clearly separate the effects of parallelization from those of structural problem decomposition. Parallelization alone, as implemented in the Flat Parallel Funnel heuristic, provides moderate reductions in wall-clock time by exploiting multiple cores, but it does not alter the underlying structure or complexity of the search space. As a result, its performance gains remain limited.

In contrast, hierarchical decomposition fundamentally reshapes the optimization problem by exploiting the spatial and structural organization of PCB layouts. By decoupling inter-cluster ordering from intra-cluster routing, the effective problem size is significantly reduced. When combined with parallel execution, these benefits compound, yielding the strongest overall performance and indicating that hierarchy and parallelism address complementary computational bottlenecks.

This efficiency gain comes at the cost of solution quality. Hierarchical heuristics consistently exhibit larger optimality gaps than flat approaches, a consequence of restricting the search space once cluster boundaries and ordering are fixed, thereby excluding certain global rearrangements of drill holes. However, this loss remains moderate and well controlled: even for the largest instances, average gaps stay below 7.5%, while runtime reductions exceed $50\times$. Moreover, gaps are measured relative to the Held–Karp lower bound, suggesting that actual solution quality may be better than reported.

Size (# Holes)	F.S. Baseline Avg. Solve Time (s)	F.P. Avg. Speedup ($\times$ times faster)	H.S. Avg. speedup ($\times$ times faster)	H.P. Avg. Speedup ($\times$ times faster)	F.S. Avg. Gap (%)	F.P. Avg. Gap (%)	H.S. Avg. Gap (%)	H.P. Avg. Gap (%)
10	0.0002	0.56	1.73	1.18	0.06	0.06	0.09	0.09
15	0.0005	1.19	2.27	2.11	0.01	0.01	1.76	1.76
20	0.0007	1.23	2.02	1.91	0.28	0.28	1.78	1.78
25	0.0010	1.39	2.42	2.46	0.53	0.52	2.76	2.76
30	0.0014	1.66	2.89	2.73	0.90	0.55	3.60	3.60
35	0.0021	1.90	3.32	3.67	1.57	1.07	4.34	4.34
40	0.0024	1.76	4.47	4.09	1.10	0.88	3.76	3.76
50	0.0045	1.94	5.80	5.87	2.08	1.43	4.89	4.89
60	0.0059	1.60	7.28	7.21	2.06	1.42	5.66	5.66
75	0.0088	1.17	10.83	10.93	2.28	1.47	6.55	6.55
90	0.0146	1.11	15.10	14.86	2.31	1.67	5.39	5.39
100	0.0191	1.00	15.66	16.11	2.75	2.24	5.54	5.54

Table 5: Heuristic performance summary for random instances. Speedups relative to Flat Sequential Pruning. Gaps relative to Held-Karp lower bound.

Size (# Holes)	F.S. Baseline Avg. Solve Time (s)	F.P. Avg. Speedup ($\times$ times faster)	H.S. Avg. speedup ($\times$ times faster)	H.P. Avg. Speedup ($\times$ times faster)	F.S. Avg. Gap (%)	F.P. Avg. Gap (%)	H.S. Avg. Gap (%)	H.P. Avg. Gap (%)
30	0.0017	1.77	2.85	2.79	0.07	0.06	1.48	1.48
50	0.0037	1.35	4.41	4.60	1.28	0.35	2.76	2.76
75	0.0079	0.96	7.78	7.43	2.24	1.13	4.87	4.87
100	0.0134	0.73	12.97	13.00	3.31	1.99	6.15	6.15

Table 6: Heuristic performance summary for structured instances. Speedups relative to Flat Sequential Pruning. Gaps relative to Held-Karp lower bound.

Performance Speedup and Optimality Gaps (Structured Instances)

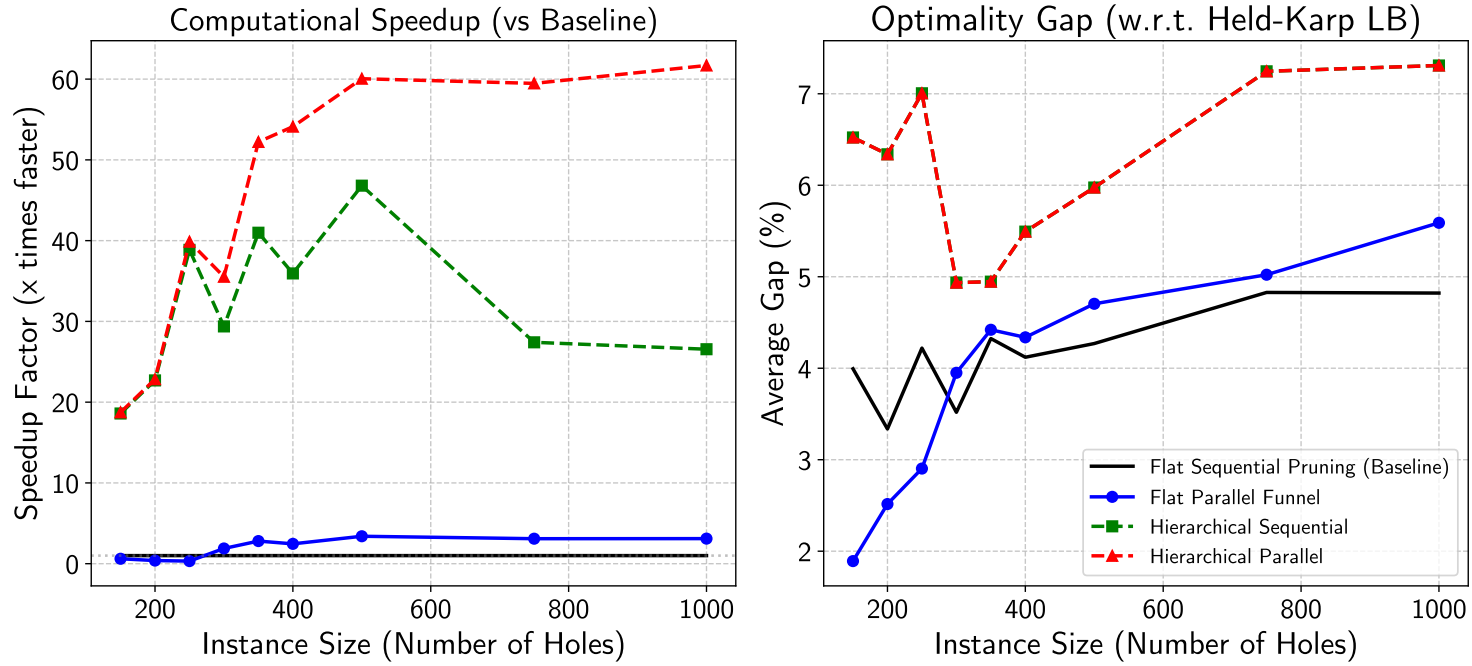


Figure 12: Heuristic performance for (big) structured instances. Speedups relative to Flat Sequential Pruning. Gaps relative to Held-Karp lower bound.

Size (# Holes)	F.S. Baseline Avg. Solve Time (s)	F.P. Avg. Speedup ($\times$ times faster)	H.S. Avg. speedup ($\times$ times faster)	H.P. Avg. Speedup ($\times$ times faster)	F.S. Avg. Gap (%)	F.P. Avg. Gap (%)	H.S. Avg. Gap (%)	H.P. Avg. Gap (%)
150	0.0383	0.56	17.13	17.48	3.99	1.89	6.52	6.52
200	0.0713	0.37	22.42	22.35	3.34	2.52	6.34	6.34
250	0.1243	0.31	34.62	36.25	4.22	2.90	7.01	7.01
300	0.1701	1.89	27.44	34.13	3.52	3.95	4.94	4.94
350	0.3515	2.69	38.52	52.16	4.32	4.42	4.95	4.95
400	0.4503	2.45	32.98	52.16	4.12	4.34	5.49	5.49
500	0.8791	3.09	40.56	59.69	4.27	4.70	5.98	5.98
750	2.1450	2.98	25.86	57.93	4.83	5.02	7.25	7.25
1000	4.6251	3.06	26.02	60.66	4.82	5.59	7.31	7.31

Table 7: Heuristic performance summary for (big) structured instances. Speedups relative to Flat Sequential Pruning. Gaps relative to Held-Karp lower bound.

5.4 Comparing SCF Solver and Our Metaheuristics

With a clear understanding of how the SCF formulation solved by CPLEX behaves as instance size increases, and of the strengths and limitations of the heuristic approaches, we can now perform a direct comparison between exact and heuristic methods.

This comparison is intentionally restricted to small instances, for which CPLEX is able to compute provably optimal solutions within reasonable time limits. Limiting the analysis to this regime allows for a fair and meaningful evaluation, where heuristic solutions can be assessed against true optima rather than against lower bounds or approximate references. The goal is not to challenge the theoretical optimality of exact methods, but to quantify how closely and how consistently heuristic approaches can approximate optimal solutions when exact benchmarks are available.

5.4.1 Qualitative Comparison of Running Times & Optimality Gaps

The results of the comparison for random instances are shown in Figure 13, whereas those of structured instances are shown in Figure 14.

Across both random and structured instances, all heuristic methods outperform CPLEX by several orders of magnitude in terms of running times. Flat heuristics already achieve speedups on the order of 10^2 and 10^3 , while hierarchical methods reach speedups of 10^4 and 10^5 as instance size increases, which is the reason why we had to use the log scale in the plots.

In terms of solution quality, the comparison remains favorable for the heuristics. The Held–Karp lower bound, used in the larger instance experiments, is already very tight for the small sizes considered here: its gap from the true optimal value computed by CPLEX never exceeds 1.5%. This confirms that the lower bound is a reliable reference point and that the optimal solutions are not significantly worse than what the bound predicts. When using the actual optimal objective as reference, the heuristic gaps shrink slightly compared to the previous analysis, remaining consistently small across both random and structured instances. This indicates that while the heuristics sacrifice some optimality in exchange for speed, they still produce solutions that are very close to the true optimum in practice.

5.4.2 Using Exact Solvers as the Comparison Baseline

Although exact solvers such as CPLEX are not viable for large-scale PCB drilling instances, their inclusion in the experimental evaluation serves two essential purposes. First, it provides a ground-truth reference for solution quality on small to medium instances, enabling an exact assessment of heuristic optimality gaps. Second, it allows the magnitude of computational savings achieved by heuristic and hierarchical methods to be quantified in absolute terms rather than solely relative to other heuristics.

The results show that while exact optimization quickly becomes computationally infeasible, the proposed hierarchical heuristics achieve several orders-of-magnitude speedups with only a modest and predictable loss in optimality, making the overall trade-off worth it.

CPLEX Exact Solver vs Heuristic Methods (Random Instances)

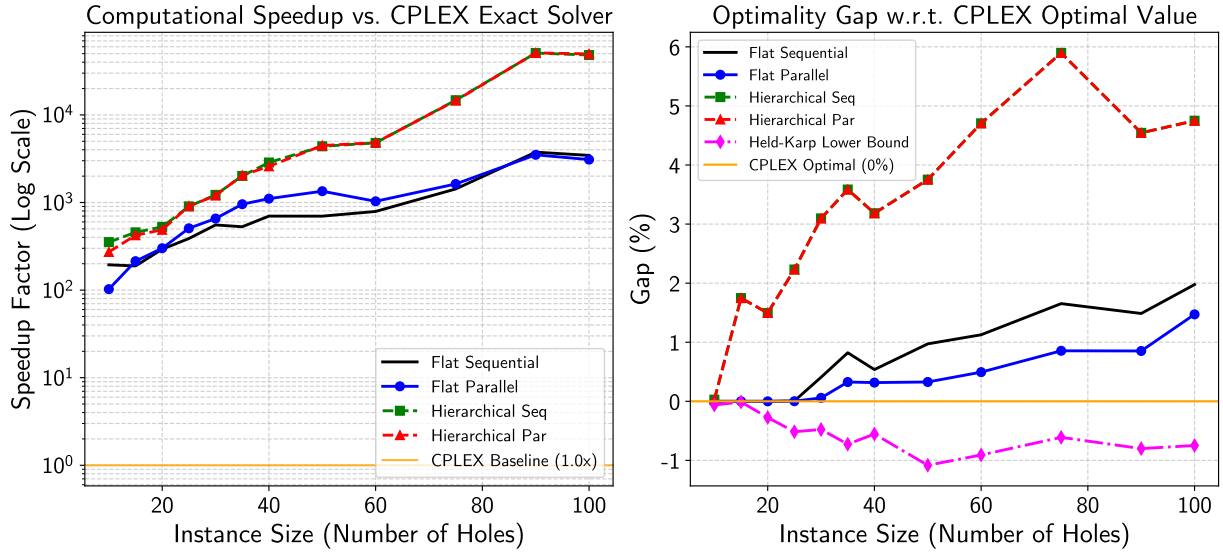


Figure 13: CPLEX SCF Solver and Heuristics comparison on random instances. Speedups relative to CPLEX SCF Solver. Gaps relative to CPLEX SCF Solver optimal objective value.

CPLEX Exact Solver vs Heuristic Methods (Structured Instances)

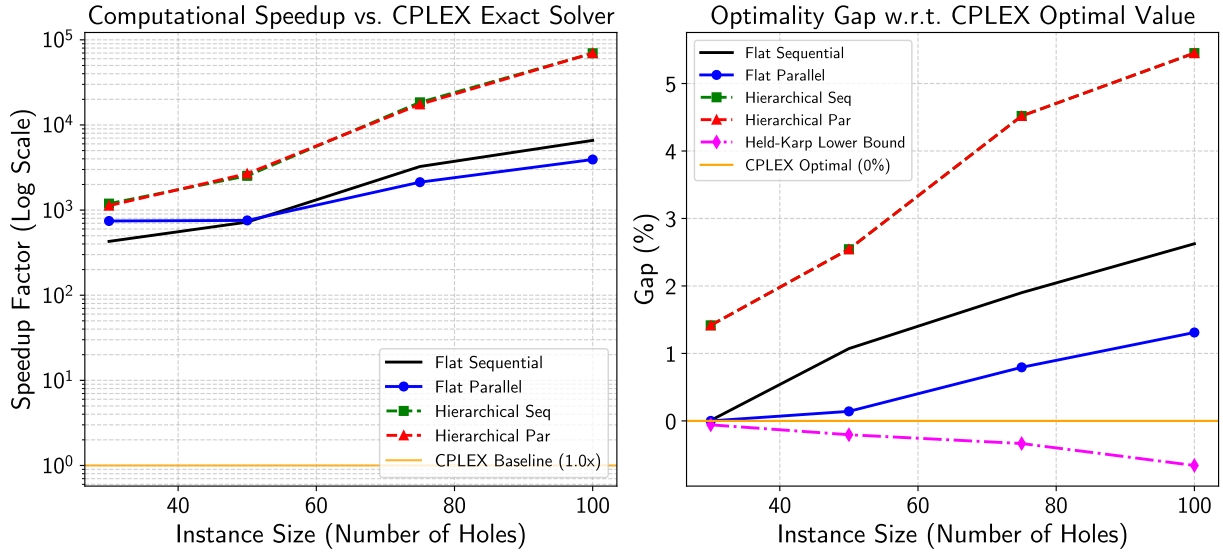


Figure 14: CPLEX SCF Solver and Heuristics comparison on structured instances. Speedups relative to CPLEX SCF Solver. Gaps relative to CPLEX SCF Solver optimal objective value.

5.5 Beyond Numbers: Tour Structure Comparison

So far, the evaluation has focused on numerical metrics such as objective value and relative gap. These indicators are useful, but they do not fully capture what the solutions actually look like in practice. In the real world, what matters is not only how small the cost is, but also how the tour behaves on the board: whether it produces reasonable paths, avoids unnecessary crossings, and respects the spatial structure of the instance.

To address this, the final section goes beyond raw numbers and compares solutions based on their tour structure. We visualize representative instances and directly compare the routes generated by the SCF solver and our heuristic algorithms. This approach offers a more operational perspective, showing how solutions would translate into real drilling patterns rather than abstract cost values.

This structural analysis is especially relevant because gap metrics can be misleading. Relative gaps are highly sensitive to the scale of the objective: when the objective value is small, even minor absolute differences can produce large percentage gaps. In our setting, where the optimal values are often already small, the resulting gaps can look worse than the actual quality difference in practice. By examining tour geometry, we can better assess whether the heuristic solutions are genuinely inferior or simply numerically different in a way that does not matter operationally.

5.5.1 Qualitative Analysis of Tours

To complement the quantitative results, Figure 15 visualizes the actual paths generated by the different solvers on a representative structured instance with 100 nodes. This visual inspection reveals the structural distinctiveness of each approach and provides intuition into the sources of their respective optimality gaps.

The exact solution produced by CPLEX exhibits a globally optimized structure, characterized by smooth transitions, minimal crossings, and subtle interleavings between spatial regions. Notice how it efficiently handles the transition between dense grids. It enters a grid, traverses it systematically, and exits at the point closest to the next logical target. This solution serves as a visual reference for optimal routing behavior.

In contrast, the Nearest Neighbour heuristic produces a highly suboptimal tour, characterized by sharp zig-zags and long, inefficient edges crossing the entire board.

Its large optimality gap reflects the algorithm’s greedy nature and inability to recover from early poor decisions, that look good locally but end up being poorly when considered in terms of the global tour.

Adding a further simple heuristic like two-opt yields enormous improvements, that allow to reduce the gap from 24% (Nearest Neighbour) to 7%. Visually, the two-opt heuristic successfully removes the long crossing edges seen in the raw nearest neighbour tour. The path looks “cleaner” and locally efficient. However, compared to CPLEX, it still lacks global coherence. While individual segments are optimized, the order in which the major components (grids) are visited remains suboptimal, legacy of the poor nearest neighbour

initialization that cannot be easily fixed. In this context, after some improvements, two-opt gets stuck in local minima.

Flat Tabu Search significantly improves tour quality by reducing crossings and eliminating many long edges. The possibility of exploring non improving solutions allows for better exploration of the solution space, which eventually yields a good solution. Nevertheless, even if the resulting tour closely resembles the optimal structure, some global rearrangements remain unexplored, leading to a small residual gap.

The Hierarchical Tabu Search explicitly exploits the clustered structure by separating inter-cluster ordering from intra-cluster routing. This yields clean and well-organized tours that closely follow the physical layout of the board. Although some global shortcuts present in the exact solution are no longer accessible due to the imposed hierarchy (cluster ordering), the resulting loss in optimality remains modest, and this comes with the huge benefit of running times that are orders of magnitude faster than the Flat Tabu Search and the exact CPLEX SCF Solver.

Overall, the figure illustrates the central trade-off explored in this work: hierarchical decomposition sacrifices a small amount of global optimality in exchange for dramatically improved scalability and computational efficiency. The visual similarity between hierarchical and optimal tours further supports the conclusion that this trade-off is well balanced and practically acceptable for PCB drilling applications, especially when real-time computation is important.

5.6 Possible Improvements

While the proposed hierarchical and flat heuristic pipeline achieves strong performance in both computational efficiency and solution quality, several avenues exist for further improvement. These opportunities span move design, parameter tuning, clustering strategies, and integration with exact methods.

5.6.1 Enhanced Local Search Operators

The current implementation relies primarily on two-opt and, while computationally cheap and effective for good improvements from a poor initial solution, sometimes it is insufficient to restructure tours at a global scale.

More expressive move operators could be introduced, including: three-opt, which would allow the solver to correct more complex crossing patterns, or or-opt, which preserves partial path structures while exploring new placements.

5.6.2 Reactive and Adaptive Parameter Tuning

The performance of pruning thresholds, tabu tenure lengths, and candidate list sizes depends strongly on instance size and structure. Currently, these parameters are either fixed or manually tuned in an approximate manner. Potential improvements include: adaptive parameter schedules, where thresholds and tenures evolve during the search based on

convergence behavior, and instance-aware tuning, using simple geometric features (e.g., number of clusters or cluster density) to select parameters dynamically.

5.6.3 Improving Clustering to Mitigate the Price of Hierarchy

The effectiveness of the hierarchical approach depends critically on the quality of the clustering. Refining the clustering stage could reduce the rigidity of the hierarchical decomposition and mitigate the loss of global optimality. As discussed in our analysis, the current “smart lookahead” aims for the nearest neighbor in the next cluster, which can be unstable if the target cluster rotates. Switching to Centroid-Based Targeting (aiming for the geometric center of the next cluster) would provide a more stable directional heuristic, preventing the solver from locking onto “outlier” nodes that result in poor entry angles.

Another advanced possibility is Cluster Boundary Relaxation, in which we introduce a post-processing step that allows nodes on the periphery of a cluster to swap into adjacent clusters. This would maintain the speed benefits of hierarchy while fixing the most glaring inefficiencies where the cluster border arbitrarily separates two nearby nodes.

5.6.4 Integrating Exact Methods

While exact solvers are impractical for large instances, they can still be leveraged strategically when the instance size is limited to small values, which is the case in most clusters for the instances that we consider. We could integrate exact solver in the pipeline to perform local exact optimization inside a cluster, or to perform the optimization of the cluster ordering, when the number of clusters is not prohibitively large.

5.6.5 Implementation-Level Improvements

Finally, additional gains may be achieved through implementation-level and parallel optimizations. Asynchronous parallel search, improved memory locality, and hardware-aware optimizations could further reduce runtime overheads, particularly for large instances. These improvements would complement the algorithmic contributions by maximizing the practical performance of the proposed framework.

Overall, these directions highlight that the proposed pipeline is not a fixed heuristic but a flexible and extensible framework. The results presented in this work establish a strong baseline, while the identified improvements provide clear paths toward reducing optimality gaps and enhancing robustness, reinforcing the suitability of hierarchical heuristic optimization for large-scale PCB drilling problems.

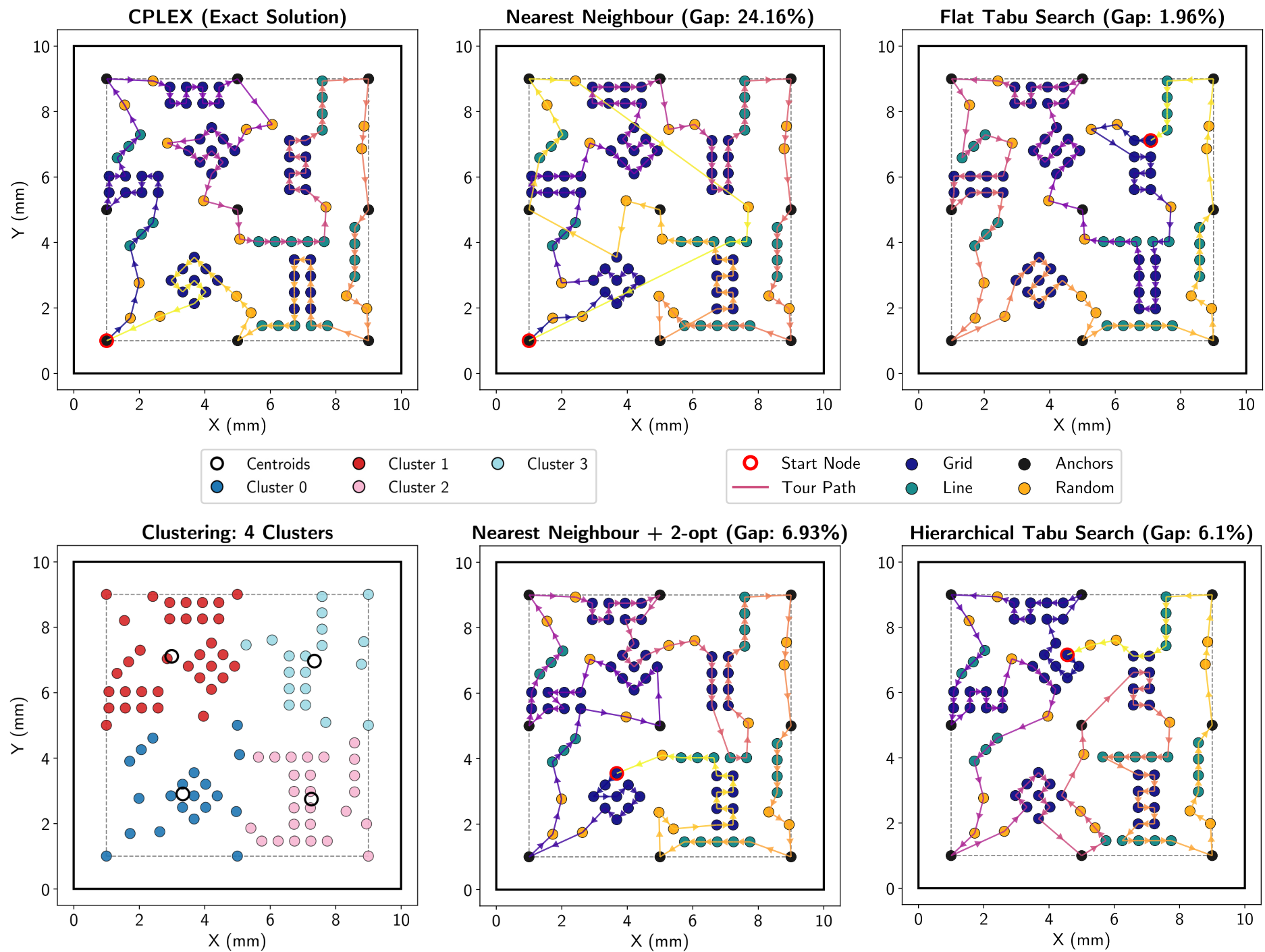


Figure 15: Tour structure comparison for different algorithms, with percentage gap to the optimal objective value of CPLEX.

5.7 Conclusions

This work set out to study the drilling path optimization problem from both an exact and a heuristic perspective, with the explicit goal of understanding not only what solutions can be obtained, but how they are obtained, how they scale, and what trade-offs they impose in practice. Along the way, we deliberately complicated our own lives by introducing multiple modeling choices, solver configurations, instance types, and algorithmic layers. While occasionally painful, this complexity was essential to expose the strengths and limitations of each approach in a controlled and meaningful way.

We began by formulating the problem as a Traveling Salesman-like model and implementing an exact solution using a Single-Commodity Flow (SCF) formulation solved by CPLEX. This provided a reliable baseline and a correctness reference. The SCF model proved robust, flexible, and fully agnostic to the semantics of the cost matrix, confirming that the formulation and implementation were sound. However, as expected, exact optimization rapidly became computationally expensive as instance size increased. Even though near-optimal solutions are often found quickly, the effort required to prove optimality dominates runtime, making exact methods impractical beyond relatively small instances.

To overcome these scalability limits, we introduced a heuristic pipeline centered around hierarchical decomposition and tabu search. Clustering allowed us to break large instances into manageable subproblems, while tabu search provided an effective local optimization mechanism within and across clusters. A significant portion of the work focused on the nontrivial task of stitching cluster-level tours into a coherent global tour. The edge-breaking strategy emerged as a key insight here: by transforming closed cluster tours into open paths, we gained the degrees of freedom needed to optimize inter-cluster connections without overconstraining the solution. While elegant in concept, the implementation reflects the inherent complexity of balancing local optimality with global consistency.

Crucially, looking beyond numerical gaps proved essential. Structural comparisons of tours revealed that many heuristic solutions, despite having non-negligible relative gaps, are operationally very close to the exact ones. In practical terms, the drill head follows clean, sensible paths that differ little from the optimal tour. This reinforces the idea that numerical optimality is not always the most meaningful metric, especially when objectives are small and gaps become inflated by normalization.

In the end, this work demonstrates that exact and heuristic methods are not competitors but complementary tools. Exact solvers provide rigor, validation, and insight into problem structure. Heuristics provide scalability, flexibility, and practical usability. By combining both, we obtain not only good solutions, but also confidence in their quality and behavior.

Before closing, an apology is in order for the sheer length of this work. It grew steadily, much like the execution times we carefully plotted, fueled by too many instances, too many metrics, and far too many hours spent staring at routes that refused to improve by even a fraction of a unit. If nothing else, this report stands as proof that a large amount of time was spent and, besides the quality of the results, we hope the effort will be appreciated.